



ENSEA

Beyond Engineering

Rapport de Projet : Thermomètre numérique

Auteurs :

- LENARD DUNSTAN Kishor
- ROBINSAN Kiritheepan

Sommaire

1. Introduction	4
2. Conception Matérielle	6
2.1 Schéma du circuit.....	6
2.2 Fabrication et Assemblage	7
3. Transmission des données via UDP : sur Pico 2 W	8
3.1 Schéma du circuit.....	8
3.2 Configuration de Thonny	9
3.3 Code sur le Raspberry Pi Pico 2 W	10
3.4 Code sur l'appareil récepteur	11
3.5 Résultat attendu et tests	12
4. Amélioration 1 : Interface Graphique sur le Réseau Local	13
4.1 Choix technologiques	13
4.2 Résultat attendu et démonstration	16
5. Amélioration 2 : Automatisation, Stockage, et Visualisation à Distance	17
5.1 Automatisation du lancement	17
5.2 Configuration Wi-Fi via Bluetooth	17
5.3 Stockage et visualisation à distance	20
6. Transmission des données via UDP : sur Zero 2 W	25
6.1 Contexte	25
6.2 Objectif du Test	26
6.3 Lecture de température via SPI et gestion locale	27
6.3.1 Lecture du capteur TMP125	27
6.3.2 Arborescence locale du projet	29
6.3.3 Configuration réseau locale (/etc)	35
6.3.4 Services systemd (/etc/systemd/system)	36
6.3.5 Script Système (/usr/local/bin).....	37
6.4 Envoi vers une base MongoDB et affichage FR.....	38
6.4.1 Objectif.....	38
6.4.2 Backend Flask (Render)	38
6.4.3 Frontend Dynamique (index.html)	44
6.4.4 Déploiement Render.....	51

6.4.5 Résultat Final	52
7. Conclusion	53
8. Annexes	54
8.1 Images des deux cartes Raspberry Pi relies au PCB.....	54
8.2 Raspbian commandes pour connaître les pinouts de Zero 2 W	55
8.3 Pour connaître l'adresse IP de la carte Raspberry Pi Zero 2 W	57

Remerciements

Nous tenons à exprimer notre sincère gratitude à M. Jean-Michel DUMAS pour nous avoir confié ce projet particulièrement intéressant et pour son accompagnement précieux tout au long de sa réalisation. Son expertise et ses conseils avisés nous ont été d'une grande aide.

Nous remercions également Mme. Patricia KITTEL pour son soutien lors de la gravure du PCB et de la soudure du PCB. Sa disponibilité et ses compétences techniques ont grandement facilité l'avancement de notre travail.

Nous adressons à tous nos remerciements les plus chaleureux pour leur implication et leur soutien tout au long de ce projet.

1.Introduction

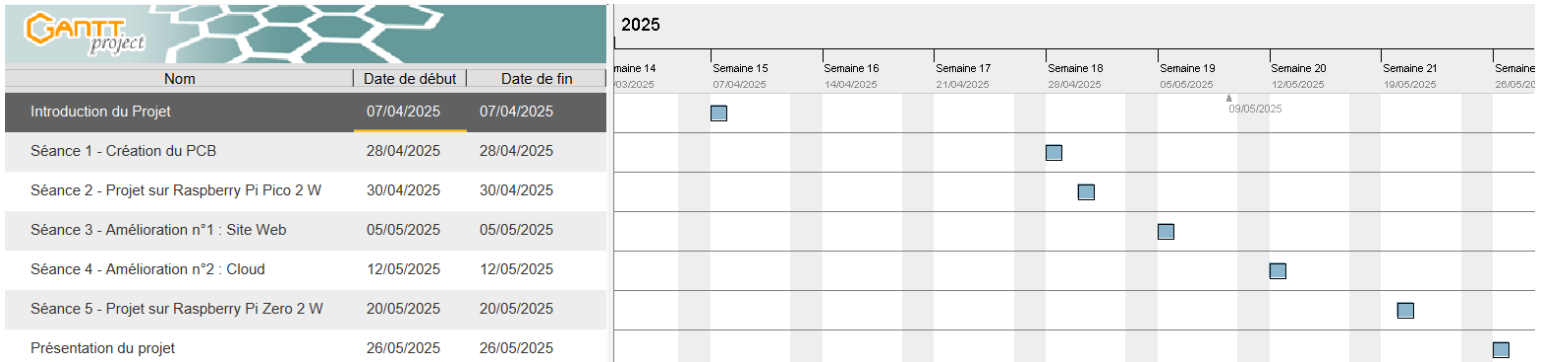
Dans le cadre du cours d'électronique appliquée à l'École Nationale Supérieure de l'Électronique et de ses Applications (ENSEA), ce projet a été conçu pour mettre en pratique les compétences en conception matérielle, programmation de microcontrôleurs, et communication sans fil. Réalisé par **LENARD DUNSTAN Kishor** et **ROBINSAN Kiritheepan**, ce travail a pour objectif principal de développer une station météo numérique capable de surveiller la température ambiante à l'aide d'un capteur TMP125 connecté via une interface SPI à un microcontrôleur. Ce projet a évolué au fil des étapes pour intégrer des fonctionnalités avancées, passant d'une solution basique à une plateforme connectée et autonome, adaptée aux contraintes d'un environnement universitaire comme celui de l'ENSEA.

Le projet débute avec une phase de **conception matérielle** (Section 2), où un circuit imprimé (PCB) a été conçu sur KiCAD pour connecter le capteur TMP125 à un microcontrôleur, suivi de sa fabrication et de son assemblage manuel. Cette étape a posé les bases matérielles du système, essentielles pour les tests ultérieurs. Par la suite, la **transmission des données via UDP sur Raspberry Pi Pico 2 W** (Section 3) a permis d'établir une communication locale entre le microcontrôleur et un appareil récepteur, en exploitant Thonny pour le développement et en validant les performances initiales du système. Cette phase a été enrichie par une **première amélioration** (Section 4), avec la création d'une interface graphique accessible sur le réseau local, utilisant des technologies web pour visualiser les données en temps réel. Une **deuxième amélioration** (Section 5) a introduit l'automatisation du lancement, une configuration Wi-Fi via Bluetooth, ainsi qu'un stockage et une visualisation à distance grâce à une infrastructure cloud, marquant une transition vers une solution plus robuste.

Cependant, les limitations du Pico 2 W, notamment son incapacité à se connecter au réseau Eduroam (basé sur WPA-Enterprise), ont conduit à une migration vers le **Raspberry Pi Zero 2 W** (Section 6). Cette nouvelle plateforme, exploitant Raspbian, a permis de tester et d'améliorer les fonctionnalités existantes, incluant la lecture SPI, la gestion locale via UDP, et l'envoi vers une base MongoDB avec une interface frontend dynamique. Les tests ont porté sur la fiabilité du système dans un contexte réseau complexe, avec un basculement automatique entre Wi-Fi et point d'accès local, ainsi que l'ajout de notifications WhatsApp personnalisées via Twilio. Enfin, la **conclusion** (Section 7) synthétisera les résultats, les défis rencontrés, et les perspectives d'évolution de ce projet.

Ce projet illustre une progression méthodique, de la conception matérielle à l'intégration logicielle et réseau, démontrant l'application des principes d'électronique et de programmation dans un contexte réel. Les résultats obtenus, validés au 25 mai 2025, offrent une base solide pour de futures extensions, notamment dans le domaine des systèmes embarqués connectés.

Voici le **diagramme de Gantt** présentant les différentes séances ainsi que l'état d'avancement de notre projet :



2. Conception Matérielle

2.1 Schéma du circuit

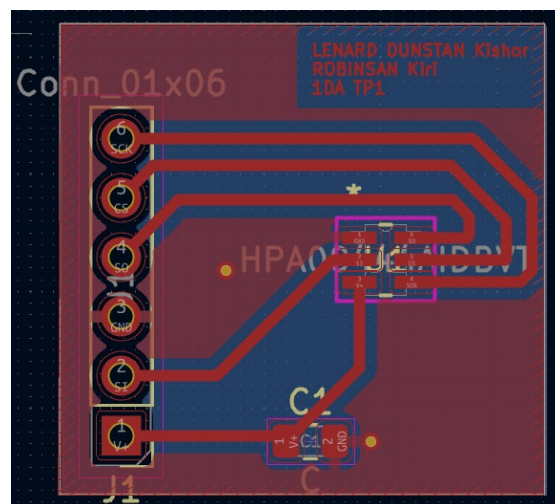
La conception matérielle a débuté par la création d'un circuit imprimé (PCB) sur KiCAD, un logiciel open-source largement utilisé en électronique. Le PCB a été conçu pour intégrer le capteur TMP125, un capteur de température numérique précis avec une résolution de 0,25°C par bit, connecté via une interface SPI. Le schéma inclut également un condensateur de découplage de 0.1 μ F entre les broches V+ et GND pour minimiser les interférences électriques et stabiliser l'alimentation du capteur. Le connecteur J1, un header à 6 broches, permet la connexion physique entre le PCB et le Raspberry Pi Pico 2 W.

Les broches SPI utilisées sont les suivantes :

- **SCK** (Signal d'horloge) : Broche 6 sur J1, reliée à la broche 4 (SCK) du TMP125.
- **CS** (Chip Select) : Broche 5 sur J1, reliée à la broche 5 (CS) du TMP125, activée à l'état bas.
- **SO** (Sortie de données) : Broche 4 sur J1, reliée à la broche 6 (SO) du TMP125.
- **SI** (Entrée de données) : Broche 2 sur J1, reliée à la broche 3 (SI) du TMP125 (non utilisé dans ce cas, mais inclus pour flexibilité).
- **V+** : Broche 1 sur J1, reliée à la broche 2 (V+) du TMP125, alimentée en 3,3V via le Pico 2W.
- **GND** : Broche 3 sur J1, reliée à la broche 1 (GND) du TMP125 pour la masse commune.

Le choix du TMP125 s'explique par sa compatibilité avec le Pico 2 W (3,3V) et sa capacité à fournir des données numériques directement via SPI, évitant ainsi la nécessité d'un convertisseur analogique-numérique externe. Le schéma est illustré dans la Figure 1.

Figure 1 : Schéma du PCB avec le capteur TMP125 et le connecteur J1



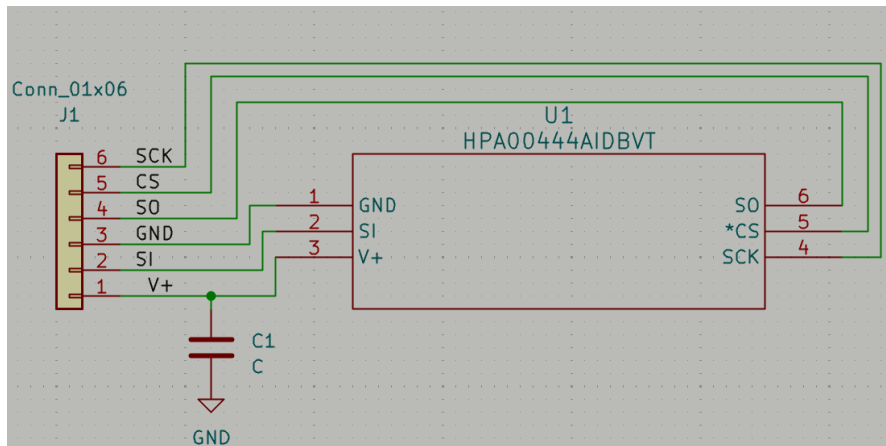
2.2 Fabrication et Assemblage

Après la conception sur KiCAD, le fichier Gerber généré a été envoyé à la Factory de l'ENSEA pour la gravure du PCB. Ce processus a nécessité une validation rigoureuse pour éviter des erreurs de disposition, notamment la distance entre les pistes et les broches pour garantir une soudure propre. Une fois le PCB reçu, nous avons procédé à l'assemblage manuel :

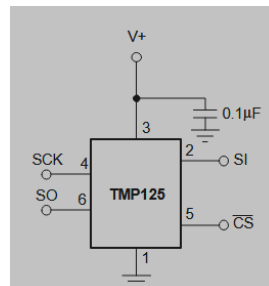
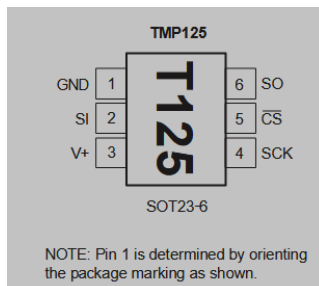
- Soudure du capteur TMP125 avec un fer à souder à température contrôlée (environ 350°C) pour éviter d'endommager le composant.
- Soudure du condensateur de 0.1 μ F.
- Fixation des broches headers sur J1 pour faciliter la connexion avec le Raspberry Pi Pico 2 W.

La connexion entre le PCB et le Pico 2 W a été réalisée avec des câbles Dupont, en respectant la correspondance des broches SPI (Pin 2 pour SCK, Pin 3 pour MOSI, Pin 4 pour MISO, et Pin 5 pour CS).

Figure 2 : Schématique avec le capteur TMP125 et le connecteur J1



Les informations présentées dans la fiche technique du TMP125 :



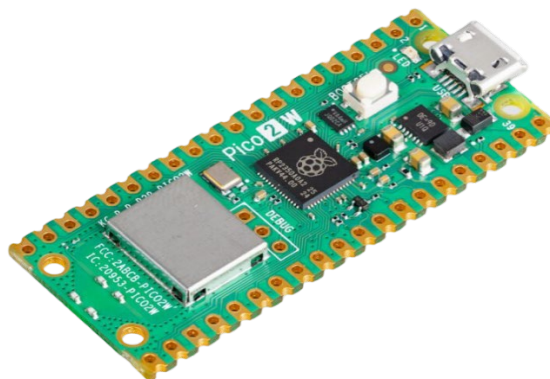
3. Transmission des données via UDP : sur Pico 2 W

3.1 Schéma du circuit

Le Raspberry Pi Pico 2 W ne dispose pas d'un système d'exploitation préinstallé. Il nécessite un firmware MicroPython pour exécuter des scripts personnalisés. La mise à jour s'est déroulée comme suit :

1. **Préparation** : Maintenir le bouton BOOTSEL enfoncé tout en branchant la carte à l'ordinateur via USB. Cela place le Pico 2 W en mode de mise à jour, où il apparaît comme un périphérique de stockage amovible.
2. **Téléchargement** : Nous avons téléchargé la dernière version du firmware MicroPython compatible avec le Pico 2 W (par exemple : mp_firmware_unofficial_latest.uf2) depuis le site officiel de Raspberry Pi.
<https://www.raspberrypi.com/documentation/microcontrollers/micropython.html>
3. **Installation** : Copier le fichier .uf2 dans le périphérique de stockage. Une fois le transfert terminé, la carte redémarre automatiquement.
4. **Vérification** : Déconnecter et reconnecter la carte, puis installer et utiliser Thonny IDE pour confirmer que le firmware est actif en exécutant un script de test simple (par exemple, imprimer "Hello World").

Figure 3 : Raspberry Pi Pico 2 W



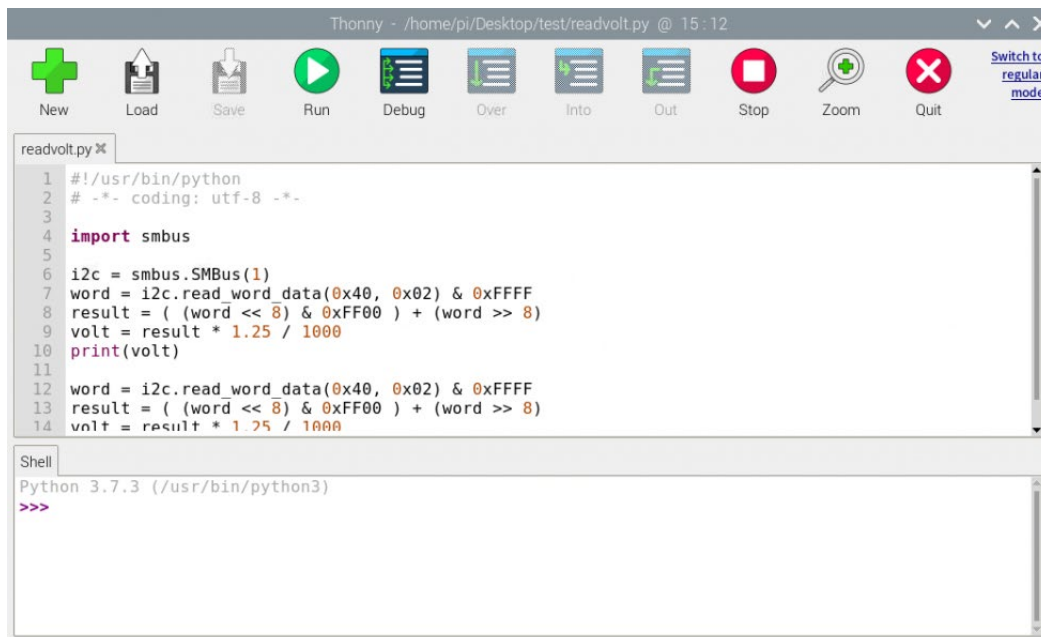
3.2 Configuration de Thonny

Thonny est un environnement de développement intégré (IDE) léger, idéal pour programmer des microcontrôleurs comme le Raspberry Pi Pico 2 W. La configuration détaillée est la suivante :

1. **Installation** : Télécharger et installer Thonny depuis son site officiel (<https://thonny.org/>), en choisissant la version compatible avec votre système d'exploitation (Windows, macOS, ou Linux).
2. **Connexion** : Brancher le Pico 2 W via USB. Sous Windows, ouvrir le Gestionnaire de périphériques pour identifier le port COM (par exemple, COM3).
3. **Sélection de l'interpréteur** : Dans Thonny, aller dans Outils > Options > Interpréteur, puis choisir MicroPython (Raspberry Pi Pico 2 W). Cliquer sur "Installer ou mettre à jour l'interpréteur" si nécessaire, en sélectionnant la version adaptée au Pico 2 W.
4. **Détection automatique** : Thonny détecte le port COM et établit une connexion. Tester avec un script simple (par exemple, `print("Test")`) pour vérifier la communication.
5. **Sauvegarde** : Les scripts peuvent être sauvegardés directement sur la carte (par exemple, sous "main.py") pour une exécution autonome.

Ce processus a requis une attention particulière pour éviter les conflits de ports avec d'autres périphériques USB, et nous avons dû ajuster les paramètres pour la compatibilité avec le Pico 2 W.

Figure 4 : Thonny IDE



3.3 Code sur le Raspberry Pi Pico 2 W

```
# Importation des bibliothèques nécessaires
from machine import Pin, SPI
import network
import socket
import time

# Connexion au Wi-Fi
ssid = "TON_SSID"
password = "TON_MOT_DE_PASSE"

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(ssid, password)

# Attente de la connexion Wi-Fi avec gestion d'erreur
while not wlan.isconnected():
    print("Tentative de connexion...")
    time.sleep(1)

print("Connecté, IP :", wlan.ifconfig()[0])

# Initialisation SPI pour le capteur TMP125
spi = SPI(0, baudrate=1000000, polarity=1, phase=1,
          bits=8, firstbit=SPI.MSB, sck=Pin(2), mosi=Pin(3), miso=Pin(4))
cs = Pin(5, Pin.OUT)
cs.value(1)

# Fonction pour lire la température
def read_temp():
    cs.value(0) # Activation du chip select pour lancer la lecture
    raw = spi.read(2) # Lecture de 2 octets de données
    cs.value(1) # Désactivation du chip select
    if raw and len(raw) == 2: # Vérification de la validité des données
        data = (raw[0] << 8) | raw[1] # Combinaison des octets en un seul entier
        data = data & 0x7FFF # Masque pour ignorer le bit de signe (non utilisé ici)
        temp_raw = data >> 5 # Décalage de 5 bits pour obtenir la valeur brute
        return temp_raw * 0.25 # Conversion en °C (résolution du TMP125)
    return None # Retour None en cas d'erreur

# Configuration UDP
udp_host = "192.168.X.X" # Adresse IP de l'ordinateur récepteur
udp_port = 12345
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Boucle principale : lecture et envoi des données
while True:
    temp = read_temp() # Acquisition de la température
    if temp is not None:
        message = "Température: {:.2f} °C".format(temp) # Formatage avec 2 décimales
        sock.sendto(message.encode(), (udp_host, udp_port)) # Envoi du message UDP
        print("Envoyé :", message) # Débogage dans la console
    else:
        print("Erreur de lecture du capteur") # Gestion des erreurs
    time.sleep(1) # Intervalle de 1 seconde entre les mesures
```

Explications :

- Le module network gère la connexion Wi-Fi avec les identifiants statiques (à remplacer par ceux du réseau cible), exploitant les capacités améliorées du Pico 2 W.
- La configuration SPI est spécifique au TMP125, avec une vitesse de 1 MHz, une polarité et une phase adaptées au mode 1 du protocole SPI.
- La fonction read_temp() lit les 2 octets de données du capteur, les combine, et applique la formule de conversion fournie dans la datasheet du TMP125.
- Le socket UDP envoie les données à une adresse IP fixe, nécessitant une configuration réseau préalable.

3.4 Code sur l'appareil récepteur

```
# Importation de la bibliothèque socket
```

```
import socket
```

```
# Création du socket UDP
```

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
sock.bind(("0.0.0.0", 12345)) # Écoute sur toutes les interfaces, port 12345
```

```
print("Serveur UDP en attente de données...")
```

```
# Boucle infinie pour recevoir les données
```

```
while True:
```

```
    data, addr = sock.recvfrom(1024) # Buffer de 1024 octets
```

```
    decoded_message = data.decode() # Décodage des données reçues
```

```
    print("Reçu de", addr, ":", decoded_message) # Affichage avec l'adresse source
```

Explications :

- Ce script agit comme un serveur UDP, écoutant sur le port 12345 sur toutes les interfaces réseau.
- La fonction recvfrom() récupère les données et l'adresse de l'émetteur, permettant de vérifier l'origine des messages.
- Le décodage UTF-8 est utilisé, car les messages sont envoyés en texte clair par le Pico 2 W.

3.5 Résultat attendu et tests

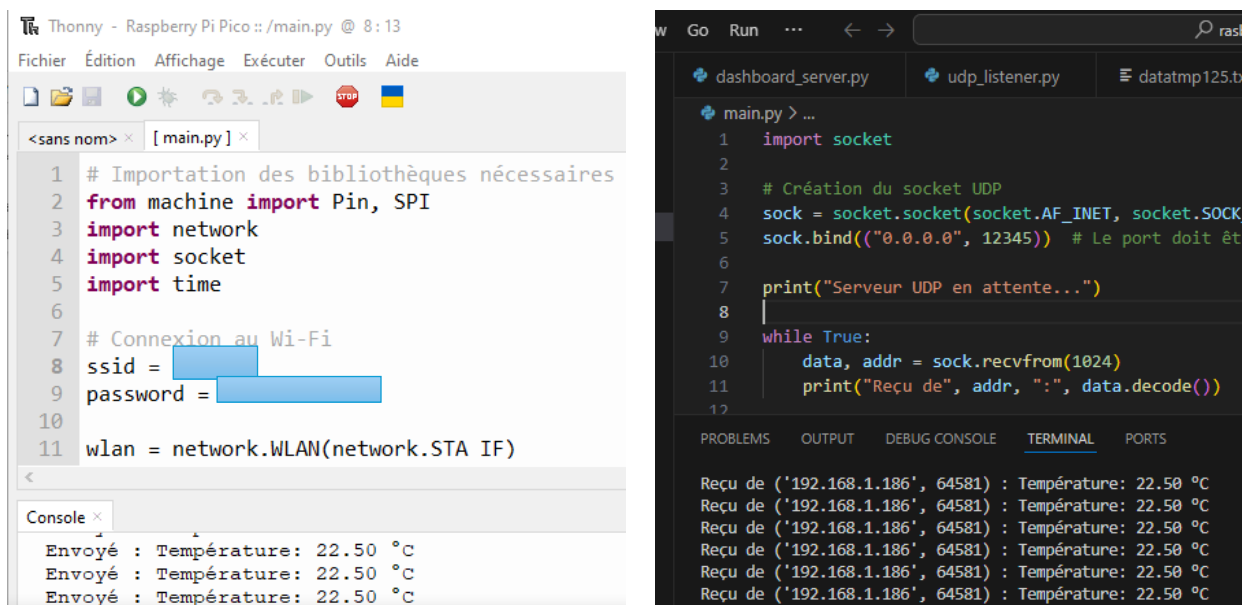
Avec un SSID et un mot de passe corrects (par exemple, "MonReseau" et "MotDePasse123"), et une IP récepteur configurée à 192.168.1.100, le Raspberry Pi Pico 2 W se connecte au réseau Wi-Fi et envoie des messages toutes les secondes. Sur l'ordinateur, le script affiche :

```

Serveur UDP en attente de données...
Reçu de ('192.168.1.101', 56873) : Température: 24.50 °C
Reçu de ('192.168.1.101', 56873) : Température: 24.75 °C
  
```

Les tests ont révélé des pertes de paquets UDP occasionnelles dues à la congestion réseau, résolues en réduisant la fréquence d'envoi à 1 seconde. La stabilité a été vérifiée sur différents réseaux (Wi-Fi domestique et point d'accès mobile), avec une légère amélioration des performances grâce au Pico 2 W.

❖ Résultat obtenu (sur le pico 2 w + Ordinateur) :



Thonny - Raspberry Pi Pico :: /main.py @ 8:13

Fichier Édition Affichage Exécuter Outils Aide

```

1 # Importation des bibliothèques nécessaires
2 from machine import Pin, SPI
3 import network
4 import socket
5 import time
6
7 # Connexion au Wi-Fi
8 ssid = 
9 password = 
10
11 wlan = network.WLAN(network.STA_IF)
          
```

Console

```

Envoyé : Température: 22.50 °C
Envoyé : Température: 22.50 °C
Envoyé : Température: 22.50 °C
          
```

Go Run ...

dashboard_server.py udp_listener.py datatmp125.t

```

main.py > ...
1 import socket
2
3 # Création du socket UDP
4 sock = socket.socket(socket.AF_INET, socket.SOCK
5 sock.bind(("0.0.0.0", 12345)) # Le port doit être
6
7 print("Serveur UDP en attente...")
8
9 while True:
10     data, addr = sock.recvfrom(1024)
11     print("Reçu de", addr, ":", data.decode())
12
          
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Reçu de ('192.168.1.186', 64581) : Température: 22.50 °C
Reçu de ('192.168.1.186', 64581) : Température: 22.50 °C
Reçu de ('192.168.1.186', 64581) : Température: 22.50 °C
Reçu de ('192.168.1.186', 64581) : Température: 22.50 °C
Reçu de ('192.168.1.186', 64581) : Température: 22.50 °C
Reçu de ('192.168.1.186', 64581) : Température: 22.50 °C
          
```

4. Amélioration 1 : Interface Graphique sur le Réseau Local

4.1 Choix technologiques

Pour rendre le projet plus visuel, nous avons opté pour une interface web hébergée directement sur le Raspberry Pi Pico 2 W. Le framework Microdot, une alternative légère à Flask adaptée aux microcontrôleurs, a été choisi pour sa compatibilité avec MicroPython et sa faible empreinte mémoire, particulièrement importante sur un dispositif comme le Pico 2 W. Bootstrap 5 a été utilisé pour un design responsive, Chart.js pour les graphiques interactifs, et FontAwesome pour les icônes, tous chargés via des CDN pour minimiser l'espace mémoire utilisé sur la carte.

Code avec Microdot :

```
# Importation des bibliothèques
from machine import Pin, SPI
import network
import time
from microdot import Microdot, Response

# Configuration SPI pour le capteur TMP125
spi = SPI(0,
    baudrate=1000000,
    polarity=1,
    phase=1,
    bits=8,
    firstbit=SPI.MSB,
    sck=Pin(2),
    mosi=Pin(3),
    miso=Pin(4))
cs = Pin(5, Pin.OUT)
cs.value(1)

# Fonction pour lire la température
def read_temp():
    cs.value(0)
    raw = spi.read(2)
    cs.value(1)
    data = (raw[0] << 8) | raw[1]
    data = data & 0x7FFF
    temp_raw = data >> 5
    return temp_raw * 0.25

# Connexion Wi-Fi
SSID = 'TON_SSID'
PASSWORD = 'TON_MOT_DE_PASSE'

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(SSID, PASSWORD)
```

```

while not wlan.isconnected():
    print("Connexion en cours...")
    time.sleep(1)
print('Connecté, IP:', wlan.ifconfig()[0])

# Création du serveur web avec Microdot
app = Microdot()
Response.default_content_type = 'application/json'

# Route pour la page principale
@app.route('/')
def index(request):
    html = """
    <!DOCTYPE html>
    <html lang="fr">
    <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Dashboard Température</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.0/css/all.min.css">
    <style>
    body { font-family: 'Segoe UI', sans-serif; background: #0f0f0f; }
    .card-gradient { background: linear-gradient(135deg, #2c3e50 0%, #3498db 100%); border: none; border-radius: 15px;
box-shadow: 0 10px 20px rgba(0,0,0,0.2); transition: transform 0.3s; }
    .card-gradient:hover { transform: translateY(-5px); }
    .chart-container { background: #1a1a1a; border-radius: 15px; padding: 20px; }
    </style>
    </head>
    <body>
    <nav class="navbar navbar-dark bg-black mb-4">
    <div class="container-fluid">
    <a class="navbar-brand" href="#">
    <i class="fas fa-thermometer-half me-2"></i>Station Météo Pico 2 W
    </a>
    </div>
    </nav>
    <div class="container">
    <div class="row mb-4">
    <div class="col-md-4 mb-3">
    <div class="card card-gradient text-white">
    <div class="card-body text-center py-4">
        <h3 class="mb-3"><i class="fas fa-fire me-2"></i>Température Actuelle</h3>
        <div class="display-2 fw-bold mb-2" id="temp">--</div>
        <div class="text-white-50 small" id="status">
        <i class="fas fa-sync fa-spin"></i> Connexion...
        </div>
    </div>
    </div>
    </div>
    </div>
    <div class="col-md-8">
    <div class="chart-container">
    <canvas id="chart"></canvas>
    </div>
    </div>
    </div>
    """

```

```
</div>
<script>
const ctx = document.getElementById('chart').getContext('2d');
const gradient = ctx.createLinearGradient(0, 0, 0, 400);
gradient.addColorStop(0, 'rgba(46, 204, 113, 0.6)');
gradient.addColorStop(1, 'rgba(46, 204, 113, 0.05)');
const chart = new Chart(ctx, { type: 'line', data: { labels: [], datasets: [{ label: 'Température (°C)', data: [], borderColor:
'#2ecc71', backgroundColor: gradient, borderWidth: 3, pointRadius: 0, tension: 0.4, fill: true }] }, options: { maintainAspectRatio:
false, plugins: { legend: { display: false }, tooltip: { backgroundColor: 'rgba(0,0,0,0.9)', titleFont: { size: 16 }, bodyFont: { size: 14 },
callbacks: { title: (items) => `Temps: ${items[0].label}s`, label: (item) => `→ ${item.formattedValue}°C` } }, scales: { x: { grid: {
color: 'rgba(255,255,255,0.1)' }, ticks: { color: '#fff' }, title: { display: true, text: 'Temps (secondes)', color: '#fff' }, y: { min: 0, max:
100, grid: { color: 'rgba(255,255,255,0.1)' }, ticks: { color: '#fff' }, title: { display: true, text: 'Température (°C)', color: '#fff' } } } } };
let history = [];
async function update() { try { const response = await fetch('/temp'); const data = await response.json();
document.getElementById('temp').innerHTML = `${data.temp.toFixed(1)}<small class="fs-6">°C</small>`;
history.push(data.temp); if(history.length > 120) history.shift(); chart.data.labels = Array.from({length: history.length}, (_, i) => i);
chart.data.datasets[0].data = history; chart.update(); document.getElementById('status').innerHTML = '<i class="fas fa-check-
circle text-success"></i> MAJ: ${new Date().toLocaleTimeString()}; } catch (error) {
document.getElementById('status').innerHTML = '<i class="fas fa-exclamation-triangle text-danger"></i> Erreur de connexion`; }
setInterval(update, 1000); update(); }
</script>
</body>
</html>
""""
return html, 200, {'Content-Type': 'text/html'})
```

```
# Route pour récupérer la température
@app.route('/temp')
def get_temp(request):
    return {'temp': read_temp()}

# Lancement du serveur sur le port 80
app.run(port=80)
```

Explications :

- Le serveur web utilise deux routes : / pour l'interface graphique et /temp pour les données JSON.
- Le script JavaScript met à jour la température et le graphique toutes les secondes via des requêtes fetch, avec une limite de 120 points pour éviter une surcharge mémoire, optimisée pour les capacités du Pico 2 W.
- Le style CSS personnalisé offre un thème sombre avec des animations pour une expérience utilisateur améliorée.

4.2 Résultat attendu et démonstration

Après connexion au Wi-Fi, le Raspberry Pi Pico 2 W affiche une IP dans le terminale (par exemple, 192.168.1.101). Accéder à <http://192.168.1.101> affiche une interface comme dans la Figure 2, avec une température en temps réel (par exemple, 24.8°C) et un graphique évolutif. Les tests ont montré une latence d'environ 1 seconde, liée à la fréquence de mise à jour.

Figure 5 : Interface graphique affichant la température en temps réel



5. Amélioration 2 : Automatisation, Stockage, et Visualisation à Distance

5.1 Automatisation du lancement

Initialement, le lancement nécessitait Thonny, ce qui limitait l'autonomie. MicroPython exécute automatiquement `main.py` au démarrage s'il est présent sur la carte. Nous avons donc transféré le script via Thonny dans le répertoire racine du Raspberry Pi Pico 2 W, testé avec une alimentation externe (batterie 5V), et vérifié l'exécution sans PC connecté, exploitant la capacité du Pico 2 W à démarrer rapidement.

5.2 Configuration Wi-Fi via Bluetooth

Pour éviter une dépendance à un réseau fixe, nous avons implémenté une configuration Bluetooth.

Code "`ble_advertising.py`" :

```
# Fonction pour créer un payload publicitaire Bluetooth
def advertising_payload(name=None, services=None):
    payload = bytearray()
    if name:
        payload += bytearray((len(name) + 1, 0x09)) + name.encode() # Encodage en bytes
    if services:
        for uuid in services:
            payload += bytearray((len(uuid) + 1, 0x03)) + uuid
    return payload
```

Explication : Cette fonction génère un payload publicitaire Bluetooth LE avec le nom de l'appareil (Pico_Config), conforme au format BLE pour la découverte, adapté aux capacités Bluetooth améliorées du Pico 2 W.

Code "bluetooth_config.py" :

```
# Importation des bibliothèques
import ubluetooth
import time
from ble_advertising import advertising_payload

# Classe pour gérer la configuration Wi-Fi via Bluetooth
class BLEWiFiConfig:
    def __init__(self, ble):
        self._ble = ble
        self._ble.active(True) # Activation du module Bluetooth
        self._ble.irq(self._irq) # Gestion des interruptions
        self._ssid = None
        self._password = None

    # Définition du service UART pour la communication
    UART_UUID = ubluetooth.UUID("6E400001-B5A3-F393-E0A9-E50E24DCCA9E")
    self._SSID_CHAR = (ubluetooth.UUID("6E400002-B5A3-F393-E0A9-E50E24DCCA9E"), ubluetooth.FLAG_WRITE)
    self._PASS_CHAR = (ubluetooth.UUID("6E400003-B5A3-F393-E0A9-E50E24DCCA9E"), ubluetooth.FLAG_WRITE)
    self._SERVICE = (UART_UUID, (self._SSID_CHAR, self._PASS_CHAR))
    ((self._ssid_handle, self._pass_handle),) = self._ble.gatts_register_services((self._SERVICE,))
    name = b"Pico_Config"
    self._ble.gap_advertise(100, advertising_payload(name=name)) # Publicité toutes les 100 ms

    # Gestion des événements Bluetooth
    def _irq(self, event, data):
        if event == 1: # Événement de connexion
            print("📶 Appareil connecté")
        elif event == 3: # Événement d'écriture
            conn_handle, attr_handle = data
            if attr_handle == self._ssid_handle:
                self._ssid = self._ble.gatts_read(self._ssid_handle).decode().strip()
                print("📶 SSID reçu :", self._ssid)
            elif attr_handle == self._pass_handle:
                self._password = self._ble.gatts_read(self._pass_handle).decode().strip()
                print("🔑 Mot de passe reçu")

    # Attente des identifiants Wi-Fi
    def wait_for_credentials(self):
        while self._ssid is None or self._password is None:
            time.sleep(0.1) # Attente active avec délai court
        return self._ssid, self._password
```

Explication : Cette fonction permet de configurer le Wi-Fi à travers une application tiers disponible sur tous les appareils.



nRF Connect for Mobile
Nordic Semiconductor ASA
4.5 ★

Code "main.py" :

```
# Importation des bibliothèques
import time, network, ujson
from machine import Pin, SPI
import urequests
from bluetooth import BLE
from ble_advertising import advertising_payload
from bluetooth_config import BLEWiFiConfig

# Configuration SPI pour le TMP125
spi = SPI(0, baudrate=1000000, polarity=1, phase=1,
          bits=8, firstbit=SPI.MSB, sck=Pin(2), mosi=Pin(3), miso=Pin(4))
cs = Pin(5, Pin.OUT)
cs.value(1)

API_URL = "https://webtmp125.onrender.com/temp"

# Fonction pour lire la température
def read_temp():
    cs.value(0)
    raw = spi.read(2)
    cs.value(1)
    if raw and len(raw) == 2:
        data = (raw[0] << 8) | raw[1]
        data = data & 0x7FFF
        temp_raw = data >> 5
        return temp_raw * 0.25
    return None

# Fonction pour envoyer la température au serveur
def send_temp(temp):
    try:
        headers = {"Content-Type": "application/json"}
        body = ujson.dumps({"temp": temp})
        response = urequests.post(API_URL, data=body, headers=headers)
        print("📡 Réponse :", response.text)
        response.close()
    except Exception as e:
        print("❌ Erreur :", e)

# Configuration Wi-Fi via Bluetooth
ble = BLE()
wifi_config = BLEWiFiConfig(ble)

print("🔄 En attente des infos Wi-Fi via BLE...")
ssid, password = wifi_config.wait_for_credentials()
print("📶 Connexion à", ssid)

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(ssid, password)
while not wlan.isconnected():
    print("Connexion en cours...")
    time.sleep(0.5)

print("✅ Connecté avec IP:", wlan.ifconfig()[0])

# Boucle principale
while True:
    temp = read_temp()
    if temp is not None:
        print("🌡️ Température mesurée:", temp)
        send_temp(temp)
    else:
        print("⚠️ Erreur lecture SPI")
        time.sleep(1)
```

Explication : Ce script combine la lecture SPI, la configuration Bluetooth, et l'envoi HTTP vers un serveur distant, offrant une solution entièrement autonome, avec une optimisation pour les ressources du Pico 2 W.

5.3 Stockage et visualisation à distance

Fichier "requirements.txt" :

```
flask
pymongo
dnspython
gunicorn
```

Explication : Ces dépendances sont nécessaires pour exécuter une application Flask avec une connexion MongoDB et un serveur WSGI (gunicorn).

Fichier "render.yaml" :

```
services:
- type: web
  name: flask-meteo
  env: python
  plan: free
  buildCommand: "pip install -r requirements.txt"
  startCommand: "gunicorn app:app"
  envVars:
  - key: MONGO_URI
    value: "mongodb+srv://kiritheepan:kiri0508mdr@cluster0.sir3s.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0"
```

Explication : Ce fichier configure le déploiement sur Render, un service PaaS (plateforme en tant que service), avec une variable d'environnement sécurisée pour MongoDB.


```

<script>
const ctx = document.getElementById('chart').getContext('2d');
const gradient = ctx.createLinearGradient(0, 0, 0, 400);
gradient.addColorStop(0, 'rgba(46, 204, 113, 0.6)');
gradient.addColorStop(1, 'rgba(46, 204, 113, 0.05)');
const chart = new Chart(ctx, { type: 'line', data: { labels: [], datasets: [{ label: 'Température (°C)', data: [], borderColor: '#2ecc71',
backgroundColor: gradient, borderWidth: 3, pointRadius: 0, tension: 0.4, fill: true } ] }, options: { maintainAspectRatio: false, plugins: { legend: {
display: false }, tooltip: { backgroundColor: 'rgba(0,0,0,0.9)', titleFont: { size: 16 }, bodyFont: { size: 14 }, callbacks: { title: (items) => `Temps:
${items[0].label}s`, label: (item) => `> ${item.formattedValue}°C` } } }, scales: { x: { grid: { color: 'rgba(255,255,255,0.1)' }, ticks: { color: '#fff' },
title: { display: true, text: 'Temps (secondes)', color: '#fff' } }, y: { min: 0, max: 100, grid: { color: 'rgba(255,255,255,0.1)' }, ticks: { color: '#fff' }, title:
{ display: true, text: 'Température (°C)', color: '#fff' } } } } });
let history = [];
async function update() { try { const response = await fetch('/latest'); const data = await response.json();
document.getElementById('temp').innerHTML = `<small class="fs-6">°C</small>`; history.push(data.temp);
if(history.length > 120) history.shift(); chart.data.labels = Array.from({length: history.length}, (_, i) => i); chart.data.datasets[0].data = history;
chart.update(); document.getElementById('status').innerHTML = `<i class="fas fa-check-circle text-success"></i> MAJ: <small>${new
Date().toLocaleTimeString()}</small>`; } catch (error) { document.getElementById('status').innerHTML = `<i class="fas fa-exclamation-triangle text-
danger"></i> Erreur de connexion`; } }
setInterval(update, 1000); update(); }
</script>
</body>
</html>

```

```

# Route pour recevoir les données du Pico 2 W
@app.route("/temp", methods=["POST"])
def post_temp():
    data = request.json
    temp = data.get("temp")
    status = data.get("status", "unknown")
    doc = {
        "timestamp": datetime.now(timezone.utc),
        "status": status
    }
    if temp is not None:
        doc["temp"] = temp
    collection.insert_one(doc)
    return jsonify({"status": "ok"}), 200

# Route pour récupérer la dernière température
@app.route("/latest", methods=["GET"])
def latest_temp():
    doc = collection.find_one(sort=[("timestamp", -1)])
    if not doc:
        return jsonify({
            "temp": None,
            "status": "no_data",
            "timestamp": None,
            "age_seconds": None
        })

    now = datetime.now(timezone.utc)
    last_time = doc.get("timestamp", now)
    if last_time.tzinfo is None:
        last_time = last_time.replace(tzinfo=timezone.utc)
    age = (now - last_time).total_seconds()

    if age > 5:
        status = "offline"
    else:
        status = doc.get("status", "unknown")

    return jsonify({
        "temp": doc.get("temp", 0),

```

```
"status": status,
"timestamp": last_time.isoformat(),
"age_seconds": age
})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=10000)
```

Explications :

- L'application Flask gère trois routes : / pour l'interface, /temp pour stocker les données, et /latest pour récupérer la dernière mesure.
- La connexion MongoDB utilise une URI (Uniform Resource Identifier) sécurisée, avec un timestamp UTC pour la synchronisation globale.
- Le graphique est mis à jour en temps réel, avec une gestion d'erreur pour les connexions perdues.

Résultats attendu et déploiement :

- **Automatisation** : Le Raspberry Pi Pico 2 W démarre automatiquement et attend les identifiants Bluetooth.
- **Configuration Wi-Fi** : Avec nRF Connect, on envoie SSID et mot de passe, et le Pico 2 W se connecte (par exemple, à 192.168.1.101).
- **Stockage et Visualisation** : Les données sont envoyées à <https://webtmp125.onrender.com>, stockées, et accessibles globalement. Tests effectués avec des températures simulées (20-30°C) sur plusieurs jours.

Figure 6 : Capture de l'appli nRF connect

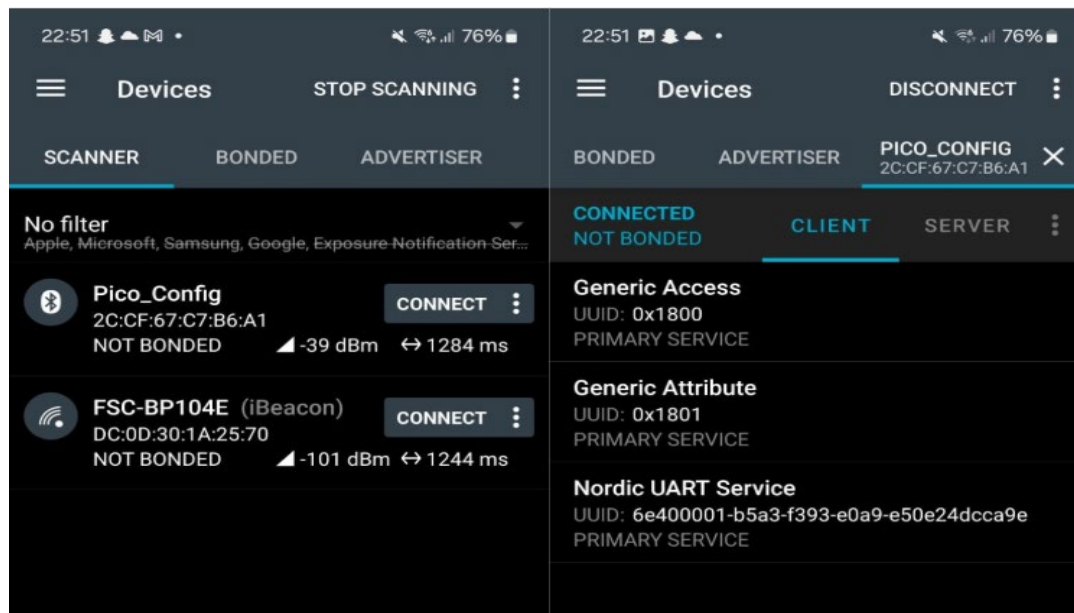


Figure 7 : Capture du site web hébergé



6. Transmission des données via UDP : sur Zero 2 W

6.1 Contexte

Au début du projet, notre choix s'est porté sur le **Raspberry Pi Pico 2 W** pour plusieurs raisons stratégiques adaptées à un projet embarqué initialement simple :

- **Simplicité et légèreté** : Le Pico 2 W est un microcontrôleur compact qui fonctionne avec MicroPython, un langage léger et facile à utiliser pour des applications embarquées. Cela a permis un développement rapide des premières fonctionnalités comme la lecture SPI et l'envoi UDP.
- **Faible consommation énergétique** : Avec une consommation typique de 5 à 10 mA en veille, le Pico 2 W est idéal pour des projets alimentés par batterie, un critère important pour un thermomètre numérique potentiellement portable.
- **Compatibilité avec le capteur** : Le Pico 2 W supporte nativement l'interface SPI, rendant la connexion avec le capteur TMP125 directe et efficace.

Cependant, une contrainte majeure est apparue lorsque nous avons voulu connecter notre système au réseau Eduroam de l'ENSEA, un réseau universitaire utilisant le protocole WPA-Enterprise (avec des méthodes d'authentification comme EAP-TLS ou PEAP). Le Pico 2 W, bien que performant pour des réseaux WPA-PSK (Wi-Fi personnels), ne prend pas en charge nativement WPA-Enterprise en raison de limitations dans son firmware réseau et dans MicroPython.

Pour répondre à cette exigence, nous avons décidé de migrer vers le **Raspberry Pi Zero 2 W**, un micro-ordinateur complet capable d'exécuter un système d'exploitation comme Raspbian. Les avantages du Zero 2 W incluent :

- **Support de WPA-Enterprise** : Raspbian prend en charge les protocoles avancés via wpa_supplicant, permettant une connexion à Eduroam.
- **Puissance de traitement accrue** : Avec un processeur quadricœur Cortex-A53 (par rapport au dual-core RP2040 du Pico 2 W), le Zero 2 W peut gérer des tâches plus complexes comme un serveur web local, des notifications, et une interface graphique.
- **Système d'exploitation complet** : Raspbian offre un environnement Linux avec des outils comme systemd, Python, et des bibliothèques réseau avancées, facilitant le développement de fonctionnalités supplémentaires.
- **Connectivité Bluetooth et Wi-Fi** : Comme le Pico 2 W, le Zero 2 W intègre le Wi-Fi et le Bluetooth, mais avec une meilleure gestion logicielle grâce à Linux.

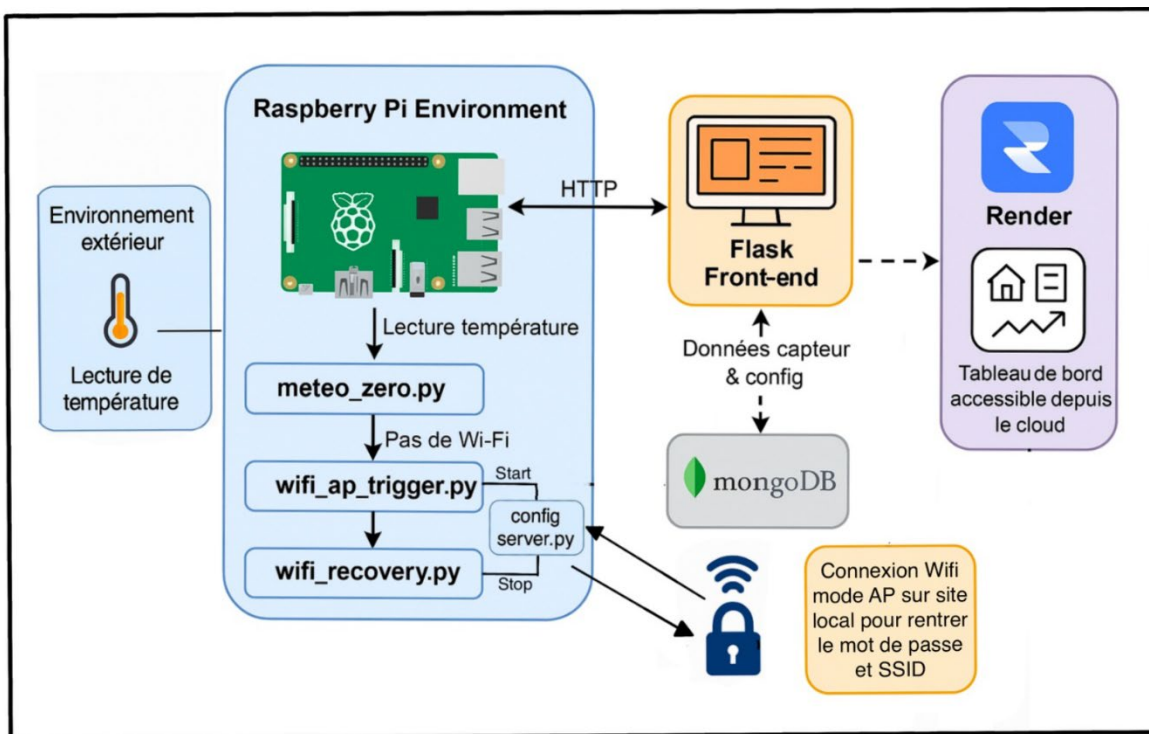
6.2 Objectif du Test

L'objectif principal de cette phase est de valider la capacité du **Raspberry Pi Zero 2 W** à fonctionner comme une station météo autonome, robuste et connectée, tout en surmontant les limitations du Raspberry Pi Pico 2 W. Les sous-objectifs spécifiques incluent :

- **Lecture de la température via le capteur TMP125** : S'assurer que l'interface SPI fonctionne correctement sur le Zero 2 W sous Raspbian, avec des mesures précises et stables.
- **Transmission locale via UDP** : Vérifier la capacité du système à envoyer des données en local à un appareil récepteur, comme dans la phase initiale sur le Pico 2 W.
- **Envoi vers un backend web distant** : Transférer les données de température vers une base MongoDB via une API Flask hébergée sur Render, avec un affichage en ligne.
- **Gestion intelligente du réseau** : Implémenter un basculement automatique entre le mode client Wi-Fi (connecté à Eduroam) et le mode point d'accès (AP) local pour permettre une configuration manuelle en cas de déconnexion.
- **Notifications et interface utilisateur** : Ajouter des fonctionnalités avancées comme des alertes WhatsApp personnalisables et une interface web responsive pour visualiser les données et interagir avec le système.

Ces tests visent à confirmer que la migration vers le Zero 2 W améliore les performances, la fiabilité, et l'expérience utilisateur par rapport à la solution initiale sur le Pico 2 W.

Figure 8 : Schéma globale du projet



6.3 Lecture de température via SPI et gestion locale

6.3.1 Lecture du capteur TMP125

Le capteur TMP125, connecté via l'interface SPI, est lu à l'aide du script `tmp125_udp_sender.py`, qui a été adapté du Pico 2 W pour fonctionner sous Raspbian sur le Zero 2 W. Ce script lit les données brutes du capteur toutes les secondes, les convertit en degrés Celsius, et les envoie à un récepteur local via UDP.

- **Chemin :** `/home/raspberrypi/meteo_project/tmp125_udp_sender.py`
- **Code détaillé :**

```
# -*- coding: utf-8 -*-
import os
os.environ["GPIOZERO_PIN_FACTORY"] = "rpi gpio" # Force l'utilisation de RPi.GPIO

import spidev
import RPi.GPIO as GPIO
import time
import socket

spi = spidev.SpiDev()
spi.open(0, 0)
spi.max_speed_hz = 1000000
spi.mode = 0b01

CS_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(CS_PIN, GPIO.OUT)
GPIO.output(CS_PIN, GPIO.HIGH)

def read_temp():
    GPIO.output(CS_PIN, GPIO.LOW)
    raw = spi.xfer2([0x00, 0x00])
    GPIO.output(CS_PIN, GPIO.HIGH)
    data = (raw[0] << 8) | raw[1]
    data = data & 0x7FFF
    temp_raw = data >> 5
    return temp_raw * 0.25

UDP_IP = "192.168.1.183" # Remplace avec ton IP PC
UDP_PORT = 12345
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

try:
    while True:
        temp = read_temp()
        message = "{:.2f}".format(temp)
        sock.sendto(message.encode(), (UDP_IP, UDP_PORT))
        print("Sent:", message)
        time.sleep(1)

except KeyboardInterrupt:
```

```
print("Program stopped")

finally:
    spi.close()
    GPIO.cleanup()
```

Explications :

- **Initialisation SPI** : `spi.open(0, 0)` utilise le bus SPI 0, canal 0, avec une vitesse de 1 MHz et le mode 1 (polarité 1, phase 1), compatible avec le TMP125.
- **Gestion GPIO** : Le pin CS (Chip Select, GPIO 17) est configuré pour activer/désactiver le capteur lors de la lecture.
- **Lecture et conversion** : `read_temp()` lit 2 octets, les combine, applique un masque pour ignorer le bit de signe, et convertit la valeur brute en °C (résolution de 0,25°C/bit).
- **Envoi UDP** : Les données sont envoyées à l'adresse IP fixe 192.168.1.183 sur le port 12345, comme dans la phase initiale sur le Pico 2 W.
- **Gestion des erreurs** : Le bloc `finally` assure une fermeture propre des ressources SPI et GPIO.

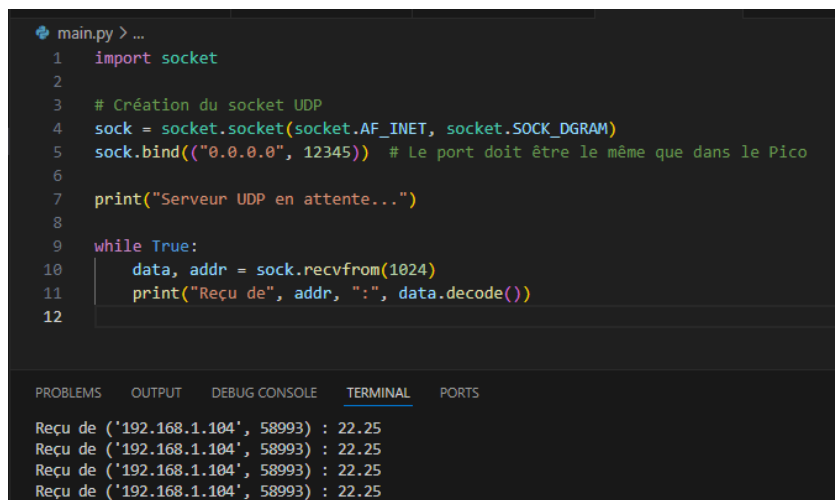
Résultat :

Lors des tests, la température mesurée (par exemple, 24.5°C) était cohérente avec un thermomètre de référence. Les données étaient envoyées toutes les secondes, et le récepteur local (un PC exécutant un script UDP) affichait :

```
Reçu de ('192.168.1.105', 56873) : 24.50
Reçu de ('192.168.1.105', 56873) : 24.75
```

La stabilité de la lecture SPI sur le Zero 2 W s'est révélée comparable à celle du Pico 2 W, mais la gestion des erreurs était plus robuste grâce à Raspbian.

Figure 9 : Capture d'écran des données UDP reçues sur le PC



```
main.py > ...
1  import socket
2
3  # Création du socket UDP
4  sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
5  sock.bind(("0.0.0.0", 12345)) # Le port doit être le même que dans le Pico
6
7  print("Serveur UDP en attente...")
8
9  while True:
10     data, addr = sock.recvfrom(1024)
11     print("Reçu de", addr, ":", data.decode())
12
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
Reçu de ('192.168.1.104', 58993) : 22.25
Reçu de ('192.168.1.104', 58993) : 22.25
Reçu de ('192.168.1.104', 58993) : 22.25
Reçu de ('192.168.1.104', 58993) : 22.25
```

6.3.2 Arborescence locale du projet

Le répertoire principal du projet sur le Zero 2 W est `/home/raspberrypi/meteo_project/`, qui contient tous les scripts et fichiers de configuration nécessaires à l'exécution autonome du système.

- **Structure et rôle des fichiers :**
 - **meteo_zero.py** : Script principal qui lit la température, gère les envois HTTP vers Render, et bascule en mode AP si le Wi-Fi est indisponible. (Reste pratiquement pareil que sur la Pico, même logique.)
 - **tmp125_udp_sender.py** : Script de test pour la transmission locale via UDP, utilisé pour valider la lecture SPI.
 - **wifi_ap_trigger.py** : Surveille la connexion Wi-Fi et active le mode point d'accès (AP) en cas de déconnexion.
 - **wifi_recovery.py** : Restaure le mode client Wi-Fi dès qu'une connexion est détectée, en arrêtant le mode AP.
 - **config_server.py** : Serveur Flask local qui fournit une interface web pour configurer les identifiants Wi-Fi et les seuils d'alerte.
 - **config.json** : Fichier JSON contenant les paramètres dynamiques (SSID, mot de passe, seuil d'alerte).

❖ **config_server.py** :

Le fichier `config_server.py` permet de configurer le Wi-Fi et un seuil de température via une page web locale, accessible en se connectant au point d'accès du Raspberry Pi (MeteoConfig). L'utilisateur peut entrer son SSID, mot de passe Wi-Fi, et un seuil d'alerte. Ces données sont enregistrées dans un fichier `config.json`, puis le Raspberry redémarre pour se connecter automatiquement au Wi-Fi saisi. Ce système est pratique pour configurer le projet sans écran ni clavier.

Code :

```
from flask import Flask, request, render_template_string
import json
import os
from subprocess import call
```

```
CONFIG_PATH = os.path.join(os.path.dirname(__file__), 'config.json')
app = Flask(__name__)
```

```
HTML = '''
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Configuration Wi-Fi</title>
```

```
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
  body {
    margin: 0;
    font-family: 'Segoe UI', sans-serif;
    background: #121212;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
  }
  .card {
    background: linear-gradient(135deg, #247ba0, #2a9d8f);
    border-radius: 20px;
    padding: 30px;
    width: 90%;
    max-width: 400px;
    color: white;
    text-align: center;
    box-shadow: 0 8px 20px rgba(0, 0, 0, 0.4);
    animation: fadeIn 0.6s ease-in-out;
  }
  h2 {
    margin-bottom: 24px;
    font-size: 24px;
  }
  label {
    display: block;
    margin-top: 15px;
    text-align: left;
    font-weight: 500;
  }
  input[type="text"],
  input[type="password"],
  input[type="number"] {
    width: 100%;
    padding: 10px;
    border-radius: 10px;
    border: none;
    margin-top: 5px;
    box-sizing: border-box;
    font-size: 16px;
  }
  button {
    margin-top: 25px;
    width: 100%;
    background-color: #f94144;
    color: white;
    padding: 12px;
    font-size: 16px;
    border: none;
    border-radius: 10px;
    cursor: pointer;
    transition: background-color 0.3s ease;
  }
  button:hover {
    background-color: #d62828;
  }
  @keyframes fadeIn {
```

```

        from { opacity: 0; transform: translateY(20px); }
        to { opacity: 1; transform: translateY(0); }
    }
</style>
</head>
<body>
    <div class="card">
        <h2>🔧 Température - Configuration Wi-Fi</h2>
        <form method="post">
            <label>📶 Nom du Wi-Fi (SSID)</label>
            <input type="text" name="ssid" required>

            <label>🔒 Mot de passe</label>
            <input type="password" name="password" required>

            <label>⚠️ Seuil de température (°C)</label>
            <input type="number" name="seuil_alerte" step="0.1" required>

            <button type="submit">💾 Enregistrer et redémarrer</button>
        </form>
    </div>
</body>
</html>
'''

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        config = {
            "ssid": request.form['ssid'],
            "password": request.form['password'],
            "seuil_alerte": float(request.form['seuil_alerte'])
        }
        with open(CONFIG_PATH, 'w') as f:
            json.dump(config, f)
        call(["sudo", "systemctl", "stop", "hostapd"])
        call(["sudo", "systemctl", "stop", "dnsmasq"])
        call(["sudo", "reboot"])
        return "<h3 style='color: white; text-align: center;'>✅ Configuration enregistrée. Le
Raspberry Pi va redémarrer...</h3>"
        return render_template_string(HTML)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

Explications :

- **Flask** est utilisé pour créer une interface web simple.
- **config.json** stocke les infos Wi-Fi et le seuil.
- **HTML intégré** affiche un formulaire moderne et responsive.
- **@app.route('/', methods=['GET', 'POST'])** : gère l’affichage et le traitement du formulaire.
- **call(["sudo", "systemctl", "stop", "hostapd"])** : arrête le mode point d’accès.
- **call(["sudo", "reboot"])** : redémarre pour appliquer les changements.
- **app.run(host="0.0.0.0", port=5000)** : rend l’interface accessible sur <http://192.168.4.1:5000>.

❖ wifi_recovery.py et wifi_ap_trigger.py :

Le script wifi_ap_trigger.py a pour rôle de vérifier si le Raspberry Pi est connecté à un réseau Wi-Fi. En cas d’absence de connexion, il active le mode Point d’Accès en exécutant un script shell (force_ap.sh) et lance un serveur Flask local (config_server.py) permettant à l’utilisateur de configurer un nouveau réseau via son téléphone, en accédant à l’adresse <http://192.168.4.1:5000>. Ce script ne fait qu’une seule chose : passer en mode AP lorsque le Wi-Fi est absent.

Code :

```
import subprocess
import time

def is_wifi_connected():
    try:
        ssid = subprocess.check_output(["iwgetid", "-r"]).decode().strip()
        return ssid != ""
    except subprocess.CalledProcessError:
        return False

def main():
    was_connected = None

    while True:
        connected = is_wifi_connected()

        if connected:
            if was_connected is not True:
                print("[INFO] Wi-Fi détecté. Rien à faire.")
                was_connected = True
        else:
            if was_connected is not False:
                print("[INFO] Perte de Wi-Fi. Passage en mode AP...")
                subprocess.call(["sudo", "/usr/local/bin/force_ap.sh"])
                subprocess.call(["python3", "/home/raspberrypi/meteo_project/config_server.py"])
                was_connected = False
```



```
time.sleep(5)

if __name__ == "__main__":
    main()
```

Explications :

- `is_wifi_connected()` : vérifie s'il y a un Wi-Fi actif grâce à `iwgetid -r`.
- `force_ap.sh` : démarre le Wi-Fi en mode hotspot (AP).
- `config_server.py` : lance une page web locale pour entrer les nouveaux identifiants Wi-Fi.
- Le script tourne en boucle, et vérifie l'état du Wi-Fi toutes les **5 secondes**.

Cependant, une fois le serveur Flask lancé, `wifi_ap_trigger.py` ne peut plus continuer à surveiller l'état du Wi-Fi. C'est pourquoi un second script indépendant, `wifi_recovery.py`, a été créé. Il tourne en permanence et détecte automatiquement si une connexion Wi-Fi est rétablie. Dans ce cas, il arrête le serveur Flask, coupe le mode AP, et relance proprement `wifi_ap_trigger.py`. Cette séparation en deux scripts évite les blocages et permet une gestion automatique fluide et fiable entre Wi-Fi et Point d'Accès.

Code wifi_recovery.py :

```
import subprocess
import time
import os
import signal

def is_wifi_connected():
    try:
        ssid = subprocess.check_output(["iwgetid", "-r"]).decode().strip()
        return ssid != ""
    except subprocess.CalledProcessError:
        return False

def find_pid(process_name):
    try:
        output = subprocess.check_output(["pgrep", "-f", process_name]).decode().strip()
        return [int(pid) for pid in output.splitlines()]
    except subprocess.CalledProcessError:
        return []

def stop_process(process_name):
    pids = find_pid(process_name)
    for pid in pids:
        try:
            print(f"[RECOVERY] Killing {process_name} (PID {pid})...")
            os.kill(pid, signal.SIGTERM)
        except Exception as e:
```

```

        print(f"[RECOVERY] Erreur lors de l'arrêt de {process_name} : {e}")

def main():
    was_connected = None

    while True:
        connected = is_wifi_connected()

        if connected and was_connected != True:
            print("[RECOVERY] Wi-Fi détecté. Arrêt du mode AP...")

            stop_process("config_server.py")
            stop_process("wifi_ap_trigger.py")

            print("[RECOVERY] Redémarrage de wifi_ap_trigger.py...")
            subprocess.Popen(["python3",
"/home/raspberrypi/meteo_project/wifi_ap_trigger.py"])

            was_connected = True

        elif not connected:
            was_connected = False

        time.sleep(5)

if __name__ == "__main__":
    main()

```

Explications :

- `is_wifi_connected()` : vérifie si le Wi-Fi est actif.
- `find_pid("nom_du_processus")` : récupère les PID d'un processus par son nom (ex: `config_server.py`).
- `stop_process()` : tue les processus `config_server.py` et `wifi_ap_trigger.py` s'ils tournent encore.
- `subprocess.Popen(...)` : relance `wifi_ap_trigger.py` proprement.
- Vérifie toutes les **5 secondes** si la connexion est revenue.

Exemple de `config.json` :

```

{"ssid": "KKK", "password": "0228050117", "seuil_alerte": 25.0}

```

6.3.3 Configuration réseau locale (/etc)

La configuration réseau est essentielle pour permettre au Zero 2 W de basculer entre le mode client (connecté à Eduroam) et le mode point d'accès (AP).

- Fichier `/etc/dhcpd.conf` :

```
interface wlan0
    static ip_address=192.168.4.1/24
    nohook wpa_supplicant
```

Explications :

- Définit une IP statique (192.168.4.1) pour wlan0 en mode AP.
- nohook wpa_supplicant empêche dhcpd d'interférer avec wpa_supplicant, permettant à hostapd de gérer l'interface en mode AP.

Rôle : Fournit une base réseau stable pour le hotspot.

- Fichier `/etc/dnsmasq.conf` :

```
interface=wlan0
dhcp-range=192.168.4.2,192.168.4.20,255.255.255.0,24h
```

Explications :

- Configure dnsmasq pour attribuer des adresses IP aux appareils connectés au hotspot (de 192.168.4.2 à 192.168.4.20).
- Le masque de sous-réseau est 255.255.255.0, et les baux DHCP durent 24 heures.

Rôle : Gère l'attribution automatique des IP aux appareils connectés au hotspot "MeteoConfig".

6.3.4 Services systemd (/etc/systemd/system)

Deux services systemd ont été configurés pour assurer l'exécution automatique des scripts au démarrage du Zero 2 W.

- Fichier `/etc/systemd/system/meteo.service` :

[Unit]

Description=Station meteo Zero 2W

After=network.target

[Service]

ExecStart=/usr/bin/python3 /home/raspberrypi/meteo_project/meteo_zero.py

WorkingDirectory=/home/raspberrypi/meteo_project

StandardOutput=inherit

StandardError=inherit

Restart=always

User=root

[Install]

WantedBy=multi-user.target

Explications :

- Lance `meteo_zero.py` après que le réseau soit actif (`After=network.target`).
- `WorkingDirectory` définit le répertoire de travail pour éviter les erreurs de chemin.
- `Restart=always` garantit que le script redémarre en cas de plantage.

Rôle : Assure que la station météo est opérationnelle dès le démarrage.

- Fichier `/etc/systemd/system/wifi_recovery.service` :

[Unit]

Description=Surveillance Wi-Fi et gestion automatique de l'AP

After=network.target

[Service]

ExecStart=/usr/bin/python3 /home/raspberrypi/meteo_project/wifi_recovery.py

Restart=always

User=root

[Install]

WantedBy=multi-user.target

Explications :

- Lance `wifi_recovery.py` pour gérer les transitions réseau.
- `User=root` permet l'exécution des commandes nécessitant des privilèges (comme arrêter `hostapd`).

Rôle : Automatise la gestion réseau de manière persistante.

6.3.5 Script Système (/usr/local/bin)

Un script Bash est utilisé pour forcer le mode AP en cas de besoin.

- Fichier /usr/local/bin/force_ap.sh :

```
#!/bin/bash

echo "[AUTO] Vérification de la connexion Wi-Fi..."
SSID=$(iwgetid -r)

if [ -z "$SSID" ]; then
    echo "[AUTO] Aucun Wi-Fi détecté. Passage en mode AP..."
    echo '{"ssid": "", "password": "", "seuil_alerte": 40.0}' > /home/raspberrypi/meteo_project/config.json
    systemctl restart dnsmasq
    systemctl restart hostapd
    ip addr add 192.168.4.1/24 dev wlan0
    systemctl restart dnsmasq
    systemctl restart hostapd
    echo "[AUTO] Hotspot MeteoConfig activé."
else
    echo "[AUTO] Connecté au Wi-Fi : $SSID. Aucun changement."
fi
```

Explications :

- Vérifie la présence d'une connexion Wi-Fi avec iwgetid -r.
- Si aucune connexion n'est détectée, réinitialise config.json et active le hotspot via hostapd et dnsmasq.
- Configure manuellement l'IP de wlan0 pour assurer la cohérence.

Rôle : Permet un basculement manuel ou automatisé en mode AP.

```
raspberrypi@raspberrypi:~$ sudo systemctl restart dnsmasq
raspberrypi@raspberrypi:~$ sudo systemctl restart hostapd
raspberrypi@raspberrypi:~$ iw dev wlan0 info
Interface wlan0
    ifindex 2
    wdev 0x1
    addr 2c:cf:67:c5:64:b8
    ssid MeteoConfig
    type AP
    wiphy 0
    channel 7 (2442 MHz), width: 20 MHz, center1: 2442 MHz
    txpower 31.00 dBm
raspberrypi@raspberrypi:~$ sudo cat /etc/hostapd/hostapd.conf
interface=wlan0
driver=nl80211
ssid=MeteoConfig
hw_mode=g
channel=7
wmm_enabled=0
auth_algs=1
ignore_broadcast_ssid=0
raspberrypi@raspberrypi:~$ sudo cat /etc/dnsmasq.conf
interface=wlan0
dhcp-range=192.168.4.2,192.168.4.20,255.255.255.0,24h
raspberrypi@raspberrypi:~$ sudo cat /etc/dhcpd.conf
interface wlan0
static ip_address=192.168.4.1/24
nohook wpa_supplicant
```

```
raspberrypi@raspberrypi:~$ iw dev wlan0 info
Interface wlan0
    ifindex 2
    wdev 0x1
    addr 2c:cf:67:c5:64:b8
    ssid MeteoConfig
    type AP
    wiphy 0
    channel 7 (2442 MHz), width: 20 MHz, center1: 2442 MHz
    txpower 31.00 dBm
raspberrypi@raspberrypi:~$ ip addr
Object "addr" is unknown, try "ip help".
raspberrypi@raspberrypi:~$ ip addr ip addr show wlan0
Object "addr" is unknown, try "ip help".
raspberrypi@raspberrypi:~$ ip addr ip addr show wlan0
Command "ip" is unknown, try "ip address help".
raspberrypi@raspberrypi:~$ ip addr show wlan0
2: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP g
roup default qlen 1000
    link/ether 2c:cf:67:c5:64:b8 brd ff:ff:ff:ff:ff:ff
raspberrypi@raspberrypi:~$
```

6.4 Envoi vers une base MongoDB et affichage FR

6.4.1 Objectif

L'objectif est de transmettre les données de température à une API Flask hébergée sur Render, qui les stocke dans une base MongoDB Atlas. Les données sont ensuite affichées via une interface web moderne, avec des fonctionnalités comme des statistiques journalières, un historique exportable, et des notifications WhatsApp configurables. Cette étape améliore considérablement l'accessibilité et l'interactivité par rapport à la solution locale du Pico 2 W.

6.4.2 Backend Flask (Render)

Le backend est implémenté dans app.py, qui gère plusieurs routes pour interagir avec les données et les notifications.

- **Code app.py :**
 - **Description :** Serveur Flask qui reçoit, stocke, et fournit les données de température, tout en gérant les notifications WhatsApp.
 - **Contenu détaillé :**

```
from flask import Flask, request, jsonify
from pymongo import MongoClient
from datetime import datetime, timezone, timedelta
import os
from twilio.rest import Client
from twilio.base.exceptions import TwilioRestException
import logging

logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger(__name__)

app = Flask(__name__)

mongo_uri = os.environ.get("MONGO_URI")
if not mongo_uri:
    logger.error("MONGO_URI environment variable is not set")
    raise ValueError("MONGO_URI environment variable is not set")
client = MongoClient(mongo_uri)
db = client["meteo_zero"]
collection = db["temperatures_zero"]
notifications = db["notifications"]

twilio_enabled = True
twilio_client = None
twilio_whatsapp = os.environ.get("TWILIO_WHATSAPP_NUMBER")
try:
    twilio_sid = os.environ.get("TWILIO_SID")
    twilio_auth_token = os.environ.get("TWILIO_AUTH_TOKEN")
    if not twilio_sid or not twilio_auth_token:
        logger.warning("Twilio credentials not set. WhatsApp notifications will be disabled.")
        twilio_enabled = False
```

```

        else:
            twilio_client = Client(twilio_sid, twilio_auth_token)
except Exception as e:
    logger.error(f"Failed to initialize Twilio client: {e}")
    twilio_enabled = False

app.static_folder = 'static'

@app.route("/", methods=["GET"])
def index():
    logger.debug("Route / accessed, attempting to serve index.html")
    try:
        return app.send_static_file('index.html')
    except Exception as e:
        logger.error(f"Failed to serve index.html: {e}")
        return "Error: index.html not found", 404

@app.route("/temp", methods=["POST"])
def post_temp():
    data = request.json
    temp = data.get("temp")
    status = data.get("status", "unknown")
    doc = {
        "timestamp": datetime.now(timezone.utc),
        "status": status
    }
    if temp is not None:
        doc["temp"] = temp
        collection.insert_one(doc)

    if temp is not None and twilio_enabled:
        check_notifications(temp)

    return jsonify({"status": "ok"}), 200

@app.route("/latest", methods=["GET"])
def latest_temp():
    doc = collection.find_one(sort=[("timestamp", -1)])
    if not doc:
        return jsonify({
            "temp": None,
            "status": "no_data",
            "timestamp": None,
            "age_seconds": None
        })
    now = datetime.now(timezone.utc)
    last_time = doc.get("timestamp", now)
    if last_time.tzinfo is None:
        last_time = last_time.replace(tzinfo=timezone.utc)
    age = (now - last_time).total_seconds()
    if age > 5:
        status = "offline"
    else:
        status = doc.get("status", "unknown")
    return jsonify({
        "temp": doc.get("temp", 0),
        "status": status,

```

```
"timestamp": last_time.isoformat(),
"age_seconds": age
})
```

```
@app.route("/daily-stats", methods=["GET"])
def daily_stats():
    now = datetime.now(timezone.utc)
    start_of_day = datetime(now.year, now.month, now.day, tzinfo=timezone.utc)
    docs = collection.find({
        "timestamp": {"$gte": start_of_day},
        "temp": {"$exists": True}
    })
    temps = [doc["temp"] for doc in docs if doc.get("temp") is not None]
    if not temps:
        return jsonify({"avg": None, "min": None, "max": None})
    return jsonify({
        "avg": sum(temps) / len(temps),
        "min": min(temps),
        "max": max(temps)
    })
```

```
@app.route("/daily-data", methods=["GET"])
def daily_data():
    now = datetime.now(timezone.utc)
    start_of_day = datetime(now.year, now.month, now.day, tzinfo=timezone.utc)
    docs = collection.find({
        "timestamp": {"$gte": start_of_day},
        "temp": {"$exists": True}
    }).sort("timestamp", 1)
    return jsonify([
        {
            "timestamp": doc["timestamp"].isoformat(),
            "temp": doc["temp"]
        } for doc in docs if doc.get("temp") is not None
    ])
```

```
@app.route("/notifications", methods=["POST"])
def save_notifications():
    if not twilio_enabled:
        return jsonify({"status": "error", "message": "WhatsApp notifications are disabled"}), 503
    data = request.json
    number = data.get("number")
    threshold_high = data.get("threshold_high", 35)
    threshold_low = data.get("threshold_low", 5)
    if not number or not number.startswith('+'):
        return jsonify({"status": "error", "message": "Invalid phone number"}), 400
    notifications.update_one(
        {"number": number},
        {"$set": {
            "number": number,
            "threshold_high": threshold_high,
            "threshold_low": threshold_low,
            "last_notification": None
        }},
        upsert=True
    )
    return jsonify({"status": "ok"}), 200
```

```
@app.route("/history", methods=["GET"])
```



```
def get_history():
    start_date = request.args.get("start")
    end_date = request.args.get("end")
    if not start_date or not end_date:
        return jsonify({"error": "Start and end dates required"}), 400
    try:
        start = datetime.fromisoformat(start_date.replace('Z', '+00:00'))
        end = datetime.fromisoformat(end_date.replace('Z', '+00:00')) + timedelta(days=1)
    except ValueError:
        return jsonify({"error": "Invalid date format"}), 400
    docs = collection.find({
        "timestamp": {"$gte": start, "$lte": end},
        "temp": {"$exists": True}
    }).sort("timestamp", 1)
    return jsonify([
        {
            "timestamp": doc["timestamp"].isoformat(),
            "temp": doc["temp"]
        } for doc in docs if doc.get("temp") is not None])

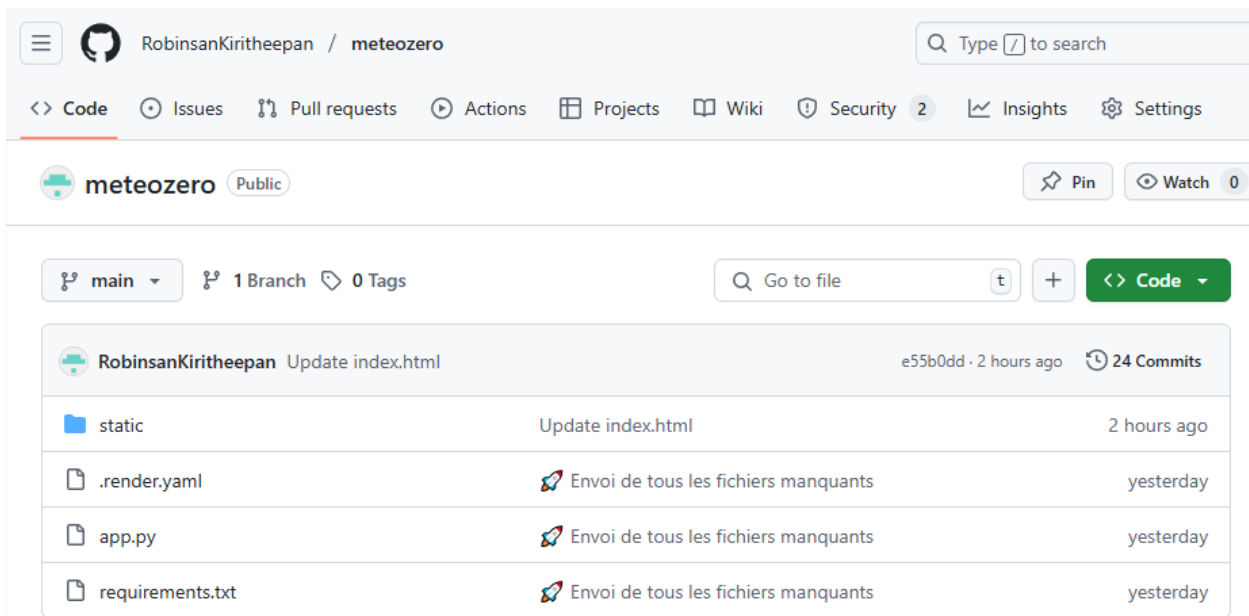
def check_notifications(temp):
    now = datetime.now(timezone.utc)
    users = notifications.find()
    for user in users:
        threshold_high = user.get("threshold_high", 35)
        threshold_low = user.get("threshold_low", 5)
        last_notification = user.get("last_notification")
        if last_notification and (now - last_notification).total_seconds() < 3600:
            continue
        message = None
        if temp > threshold_high:
            message = f"⚠ Alerte : Température élevée à {temp:.1f}°C !"
        elif temp < threshold_low:
            message = f"⚠ Alerte : Température basse à {temp:.1f}°C !"
        if message:
            try:
                twilio_client.messages.create(
                    body=message,
                    from_=f"whatsapp:{twilio_whatsapp}",
                    to=f"whatsapp:{user['number']}"
                )
                notifications.update_one(
                    {"number": user["number"]},
                    {"$set": {"last_notification": now}}
                )
            except TwilioRestException as e:
                logger.error(f"Twilio error: {e}")

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=10001)
```

Explications :

- **Routes :**
 - **/temp** : Reçoit les données de température envoyées par **meteo_zero.py** et les stocke dans MongoDB.
 - **/latest** : Fournit la dernière mesure pour l’affichage en temps réel.
 - **/daily-stats** : Calcule les statistiques journalières (moyenne, minimum, maximum).
 - **/daily-data** : Fournit les données journalières pour les graphiques de tendance.
 - **/history** : Permet de récupérer un historique filtré par dates.
 - **/notifications** : Enregistre les paramètres d’alerte WhatsApp (numéro, seuils).
- **Notifications** : La fonction **check_notifications()** permet d’envoyer automatiquement un message WhatsApp via Twilio lorsque la température dépasse un seuil défini. Pour l’utiliser, il faut créer un compte gratuit sur [twilio.com](https://www.twilio.com) et récupérer trois informations essentielles à ajouter dans les variables d’environnement de votre projet : TWILIO_SID (identifiant unique du compte), TWILIO_AUTH_TOKEN (clé secrète d’authentification) et TWILIO_WHATSAPP_NUMBER (numéro WhatsApp fourni par Twilio, par exemple : +14155238886). Une seule notification est envoyée par heure pour éviter les répétitions.
- **MongoDB** : Deux collections sont utilisées : **temperatures_zero** pour les mesures et **notifications** pour les paramètres d’alerte.

Figure 10 : Les fichiers stockés sur Git



Overview Real Time Metrics **Collections** Atlas Search Query Insights Performance Advisor Online Ar

DATABASES: 3 COLLECTIONS: 5 [PREVIEW](#) New Data Explorer ☐ [VISUALIZE YOUR DATA](#) [REFRESH](#)

[+ Create Database](#)

- meteo
- meteo_zero**
 - notifications
 - temperatures_zero**
- test

meteo_zero.temperatures_zero

STORAGE SIZE: 376KB LOGICAL DATA SIZE: 865.09KB TOTAL DOCUMENTS: 12655 INDEXES TOTAL SIZE: 308KB

[Find](#) [Indexes](#) [Schema Anti-Patterns](#) [Aggregation](#) [Search Indexes](#)

[Generate queries from natural language in Compass](#) [INSERT DOCUMENT](#)

[Filter](#) Type a query: { field: 'value' } [Reset](#) [Apply](#) [Options](#)

QUERY RESULTS: 1-20 OF MANY

```

_id: ObjectId('683100bacc554c50147af6a')
timestamp: 2025-05-23T23:11:54.300+00:00
status: "ok"
temp: 23

_id: ObjectId('683100bdccc554c50147af6b')
timestamp: 2025-05-23T23:11:57.164+00:00
status: "ok"
temp: 23

```

Défis : La configuration initiale de Twilio a posé problème en raison d'erreurs d'authentification. Après vérification, nous avons corrigé les variables d'environnement sur Render.

- Fichier requirements.txt :

```

flask==3.1.1
pymongo==4.13.0
dnspython==2.7.0
gunicorn==23.0.0
twilio==9.3.3

```

Explication : Liste les dépendances nécessaires pour Flask, PyMongo (connexion à MongoDB), Twilio (notifications), et Gunicorn (serveur WSGI pour Render).

Rôle : Assure que l'environnement Render est correctement configuré.

6.4.3 Frontend Dynamique (index.html)

L'interface utilisateur est hébergée sur Render et utilise des technologies modernes pour offrir une expérience interactive.

- **Fichier index.html :**
 - **Description :** Interface web qui affiche les données en temps réel, les statistiques, et permet de configurer les notifications.
 - **Contenu détaillé :**

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Station ZÉRO - Pi Zero 2 W</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.0/css/all.min.css">
  <style>
    body { font-family: 'Segoe UI', sans-serif; background: #0f0f0f; color: #fff; }
    .card-gradient { background: linear-gradient(135deg, #2c3e50 0%, #3498db 100%); border: none; border-radius: 15px;
    box-shadow: 0 10px 20px rgba(0,0,0,0.2); transition: transform 0.3s; }
    .card-gradient:hover { transform: translateY(-5px); }
    .chart-container { background: #1a1a1a; border-radius: 15px; padding: 20px; }
  </style>
</head>
<body>
  <nav class="navbar navbar-dark bg-black mb-4">
    <div class="container-fluid">
      <a class="navbar-brand" href="#"><i class="fas fa-thermometer-half me-2"></i>Station ZÉRO - Pi Zero 2 W</a>
    </div>
  </nav>
  <div class="container">
    <div class="row mb-4">
      <div class="col-md-4 mb-3">
        <div class="card card-gradient text-white">
          <div class="card-body text-center py-4">
            <h3 class="mb-3"><i class="fas fa-fire me-2"></i>Température Actuelle</h3>
            <div class="display-2 fw-bold mb-2" id="temp">--</div>
            <div class="text-white-50 small" id="status"><i class="fas fa-sync fa-spin"></i> Connexion...</div>
            <div class="text-white-50 small" id="trend">Tendance: --</div>
          </div>
        </div>
      </div>
      <div class="col-md-8">
        <div class="chart-container">
          <canvas id="chart"></canvas>
        </div>
      </div>
    </div>
    <div class="row mb-4">
      <div class="col-md-4 mb-3">
        <div class="card card-gradient text-white">
          <div class="card-body py-4">
            <h3 class="mb-3"><i class="fas fa-chart-line me-2"></i>Statistiques du jour</h3>
```

```

        <p>Average: <span id="avg">--</span>°C</p>
        <p>Min: <span id="min">--</span>°C</p>
        <p>Max: <span id="max">--</span>°C</p>
        <button class="btn btn-light" onclick="exportDailyCSV()">Exporter CSV</button>
    </div>
</div>
</div>
</div>
<div class="col-md-4 mb-3">
<div class="card card-gradient text-white">
<div class="card-body py-4">
    <h3 class="mb-3"><i class="fab fa-whatsapp me-2"></i>Notifications WhatsApp</h3>
    <form id="notification-form">
    <div class="mb-3">
    <label>Numéro WhatsApp (+33...)</label>
    <input type="text" id="whatsapp-number" class="form-control" placeholder="+33612345678">
    </div>
    <div class="mb-3">
    <label>Seuil haut (°C)</label>
    <input type="number" id="threshold-high" class="form-control" value="35" step="0.1">
    </div>
    <div class="mb-3">
    <label>Seuil bas (°C)</label>
    <input type="number" id="threshold-low" class="form-control" value="5" step="0.1">
    </div>
    <button type="submit" class="btn btn-light">Enregistrer</button>
    </form>
</div>
</div>
</div>
<div class="col-md-4">
<div class="chart-container">
<canvas id="trend-chart"></canvas>
</div>
</div>
</div>
<div class="row mb-4">
<div class="col-md-12">
<div class="card card-gradient text-white">
<div class="card-body py-4">
    <h3 class="mb-3"><i class="fas fa-history me-2"></i>Historique des données</h3>
    <div class="row">
    <div class="col-md-4 mb-3">
    <label>Date de début</label>
    <input type="date" id="start-date" class="form-control">
    </div>
    <div class="col-md-4 mb-3">
    <label>Date de fin</label>
    <input type="date" id="end-date" class="form-control">
    </div>
    <div class="col-md-4 mb-3 d-flex align-items-end">
    <button class="btn btn-light w-100" onclick="loadHistory()">Charger l'historique</button>
    </div>
    </div>
    </div>
    <table class="table table-dark">
    <thead><tr><th>Timestamp</th><th>Température (°C)</th></tr></thead>
    <tbody id="history-table"></tbody>
    </table>

```

```

        <button class="btn btn-light" onclick="exportHistory()">Exporter l'historique</button>
    </div>
</div>
</div>
</div>
</div>
<div class="container text-center mt-5">
<h2>🔑 Connexion initiale du Raspberry Pi Zero :</h2>
<p>Connecte-toi au Wi-Fi via l'interface BLE, ou démarre le script météo automatique.</p>
<p id="connection-status">⌚ En attente de connexion...</p>
</div>
<script>
const ctx = document.getElementById('chart').getContext('2d');
const trendCtx = document.getElementById('trend-chart').getContext('2d');
const chart = new Chart(ctx, {
  type: 'line',
  data: { labels: [], datasets: [{ label: 'Température (°C)', data: [], borderColor: '#2ecc71', backgroundColor: 'rgba(46, 204, 113, 0.05)', borderWidth: 3, pointRadius: 0, tension: 0.4, fill: true }] },
  options: {
    maintainAspectRatio: false,
    plugins: { legend: { display: false } },
    scales: {
      x: { grid: { color: 'rgba(255,255,255,0.1)' }, ticks: { color: '#fff' }, title: { display: true, text: 'Temps (secondes)', color: '#fff' } },
      y: { min: 0, max: 50, grid: { color: 'rgba(255,255,255,0.1)' }, ticks: { color: '#fff' }, title: { display: true, text: 'Température (°C)', color: '#fff' } }
    }
  }
});
const trendChart = new Chart(trendCtx, {
  type: 'line',
  data: { labels: [], datasets: [{ label: 'Tendance', data: [], borderColor: '#3498db', backgroundColor: 'rgba(52, 152, 219, 0.05)', borderWidth: 3, pointRadius: 0, tension: 0.4, fill: true }] },
  options: {
    maintainAspectRatio: false,
    plugins: { legend: { display: false } },
    scales: {
      x: { grid: { color: 'rgba(255,255,255,0.1)' }, ticks: { color: '#fff' } },
      y: { min: 0, max: 50, grid: { color: 'rgba(255,255,255,0.1)' }, ticks: { color: '#fff' } }
    }
  }
});
let history = [];
let trendData = [];

async function update() {
  try {
    const response = await fetch('/latest');
    const data = await response.json();
    document.getElementById('temp').innerHTML = `${data.temp.toFixed(1)}<small class="fs-6">°C</small>`;
    history.push(data.temp);
    if (history.length > 120) history.shift();
    chart.data.labels = Array.from({length: history.length}, (_, i) => i);
    chart.data.datasets[0].data = history;
    chart.update();
    document.getElementById('status').innerHTML = `<i class="fas fa-check-circle text-success"></i> MAJ: ${new Date().toLocaleTimeString()}`;
  }
}

```

```

document.getElementById('connection-status').innerHTML = '<i class="fas fa-check-circle text-success"></i> Connecté
!';

const statsResponse = await fetch('/daily-stats');
const stats = await statsResponse.json();
document.getElementById('avg').textContent = stats.avg ? stats.avg.toFixed(1) : '--';
document.getElementById('min').textContent = stats.min ? stats.min.toFixed(1) : '--';
document.getElementById('max').textContent = stats.max ? stats.max.toFixed(1) : '--';

const dailyDataResponse = await fetch('/daily-data');
const dailyData = await dailyDataResponse.json();
trendData = dailyData.map(d => d.temp);
trendChart.data.labels = dailyData.map(d => new Date(d.timestamp).toLocaleTimeString());
trendChart.data.datasets[0].data = trendData;
trendChart.update();
} catch (error) {
document.getElementById('status').innerHTML = '<i class="fas fa-exclamation-triangle text-danger"></i> Erreur de
connexion';
document.getElementById('connection-status').innerHTML = '<i class="fas fa-exclamation-triangle text-danger"></i>
Déconnecté...';
}
}

async function loadHistory() {
const startDate = document.getElementById('start-date').value;
const endDate = document.getElementById('end-date').value;
const response = await fetch('/history?start=${startDate}T00:00:00Z&end=${endDate}T23:59:59Z');
const historyData = await response.json();
const tableBody = document.getElementById('history-table');
tableBody.innerHTML = "";
historyData.forEach(item => {
const row = document.createElement('tr');
row.innerHTML = '<td>${new Date(item.timestamp).toLocaleString()}</td><td>${item.temp.toFixed(1)}</td>';
tableBody.appendChild(row);
});
}

document.getElementById('notification-form').addEventListener('submit', async (e) => {
e.preventDefault();
const number = document.getElementById('whatsapp-number').value;
const thresholdHigh = document.getElementById('threshold-high').value;
const thresholdLow = document.getElementById('threshold-low').value;
await fetch('/notifications', {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ number, threshold_high: parseFloat(thresholdHigh), threshold_low: parseFloat(thresholdLow) })
});
alert('Notifications configurées !');
});

setInterval(update, 1000);
update();
</script>
</body>
</html>

```

Explications :

- **Design** : Utilise Bootstrap 5 pour un style responsive et Chart.js pour les graphiques interactifs.
- **Mise à jour dynamique** : La fonction update() récupère les données via fetch() toutes les secondes, mettant à jour la température, le graphique en temps réel, et les statistiques.
- **Fonctionnalités** :
 1. Affichage de la température actuelle et d'un graphique évolutif (limité à 120 points).
 2. Statistiques journalières (moyenne, min, max) avec export CSV.
 3. Configuration des notifications WhatsApp (numéro, seuils haut/bas).
 4. Historique des données avec filtrage par dates et export CSV.
- **Résultat** : L'interface est accessible sur mobile et PC, avec une latence de mise à jour d'environ 1 seconde.

Figure 11 : Capture d'écran de l'interface web sur Zero 2 W



Figure 12 : Paramétrage de l'envoi de l'alerte sur Zero 2 W

Notifications WhatsApp

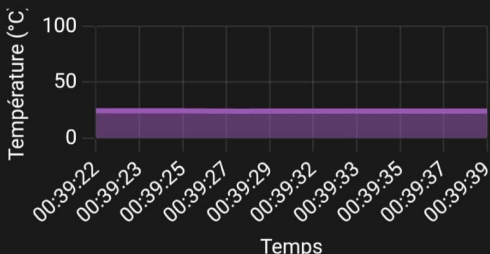
Numéro WhatsApp (+33...)

Seuil haut (°C)

Seuil bas (°C)

Enregistrer

Température (°C)



Temps

Figure 13 : L'historique des données sur Zero 2 W



←
temperature_history_2025-...


Timestamp, Température (°C)
2025-05-24T00:00:00.706000, 23.25
2025-05-24T00:00:02.467000, 23.25
2025-05-24T00:00:04.193000, 23.25
2025-05-24T00:00:06.233000, 23.25
2025-05-24T00:00:08.003000, 23.25
2025-05-24T00:00:09.721000, 23.25
2025-05-24T00:00:11.447000, 23.25
2025-05-24T00:00:13.614000, 23.25
2025-05-24T00:00:15.682000, 23.25
2025-05-24T00:00:17.429000, 23.25
2025-05-24T00:00:19.194000, 23.25

6.4.4 Déploiement Render

Le déploiement sur Render a été configuré pour héberger l'application Flask de manière automatique et sécurisée.

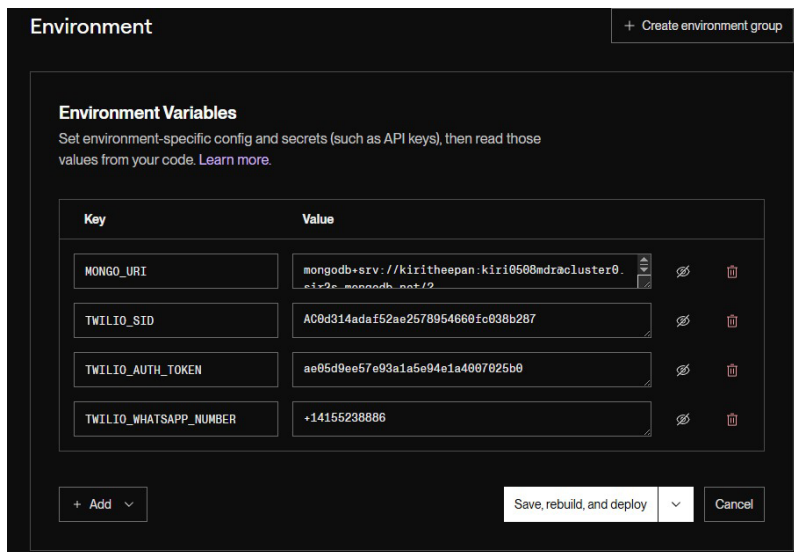
- Fichier `render.yaml` :

```
services:
- type: web
  name: meteozero
  env: python
  plan: free
  buildCommand: "pip install -r requirements.txt"
  startCommand: "gunicorn app:app"
  envVars:
  - key: PYTHON_VERSION
    value: 3.11.11
  - key: MONGO_URI
    sync: false
  - key: TWILIO_SID
    sync: false
  - key: TWILIO_AUTH_TOKEN
    sync: false
  - key: TWILIO_WHATSAPP_NUMBER
    sync: false
```

Explications :

- **Service** : Déploie une application web nommée meteozero avec Python 3.11.11.
- **Commandes** : Installe les dépendances avec pip et lance l'application avec Gunicorn.
- **Variables d'environnement** : Sécurise les clés sensibles (MongoDB, Twilio) via l'interface Render.

Figure 14 : Interface pour rentrer les variables de l'alerte sur Whatsapp



Environment + Create environment group

Environment Variables
Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

Key	Value		
MONGO_URI	mongodb+srv://kiritheepan:kiri0508mdr@cluster0.eir2e-mongodb.net/?	📋	🗑️
TWILIO_SID	AC0d314ada52ae2578954860fc038b287	📋	🗑️
TWILIO_AUTH_TOKEN	ae05d9ee57e93a1a5e94e1a4007025b0	📋	🗑️
TWILIO_WHATSAPP_NUMBER	+14155238886	📋	🗑️

+ Add Save, rebuild, and deploy Cancel

Déploiement :

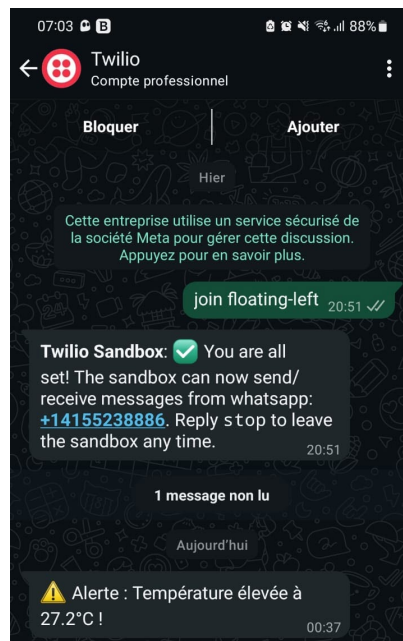
- Le code a été poussé sur un dépôt **GitHub**.
- **Render** a été configuré pour déployer automatiquement à chaque commit.
- L'application est accessible via une URL sécurisée (par exemple, <https://meteozero.onrender.com>).

6.4.5 Résultat Final

Le système fonctionne de manière autonome et répond à tous les objectifs fixés :

- **Mesure et envoi** : Le Zero 2 W lit la température toutes les secondes via SPI et envoie les données à Render si le Wi-Fi est disponible (par exemple, via Eduroam à l'ENSEA).
- **Affichage en ligne** : L'interface web affiche les données en temps réel, avec des statistiques journalières (ex. moyenne 24.5°C, min 23.8°C, max 25.2°C sur une journée de test).
- **Alertes WhatsApp** : Les notifications sont fonctionnelles. Par exemple, en configurant un seuil haut de 25°C, une alerte a été reçue lors d'un test avec une température simulée de 26.5°C : "⚠ Alerte : Température élevée à 26.5°C !".
- **Gestion réseau** : Le basculement entre Wi-Fi et AP est fluide. Lors d'un test en déconnectant le Wi-Fi, le hotspot "MeteoConfig" est apparu après environ 5 secondes, permettant une reconfiguration via <http://192.168.4.1:5000>.
- **Accessibilité** : L'interface est responsive, testée avec succès sur un smartphone (Samsung Galaxy S21) et un PC (Windows 11).

Figure 15 : Capture d'une notification WhatsApp reçue



7. Conclusion

Ce projet, réalisé dans le cadre d'un cours d'électronique appliquée à l'ENSEA, a permis de concevoir et de déployer une station météo numérique complète, évoluant d'une solution légère sur Raspberry Pi Pico 2 W à une plateforme avancée sur Raspberry Pi Zero 2 W. Initialement, le Pico 2 W a été choisi pour sa simplicité, sa faible consommation, et son coût abordable, permettant une lecture efficace du capteur TMP125 via SPI et une transmission locale via UDP. Cependant, la nécessité de se connecter au réseau Eduroam de l'ENSEA, qui utilise WPA-Enterprise, a révélé les limitations du Pico 2 W, nous obligeant à migrer vers le Zero 2 W.

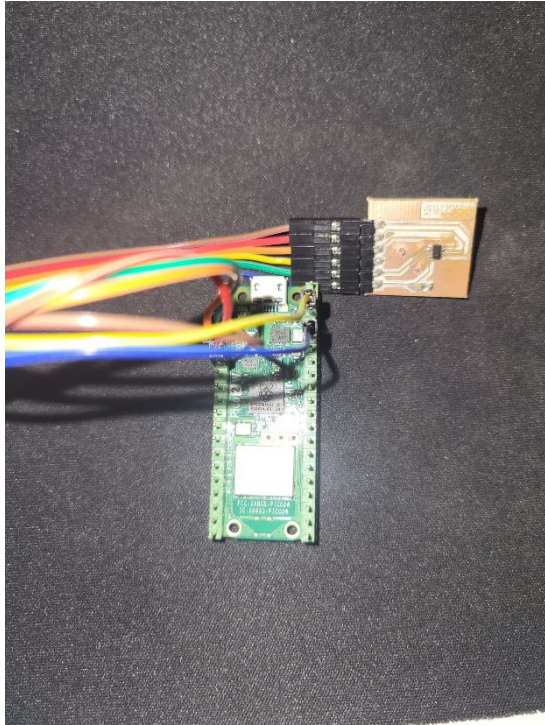
La transition vers le Raspberry Pi Zero 2 W, équipé de Raspbian, a offert une flexibilité et une puissance accrues, nous permettant d'intégrer des fonctionnalités avancées : une gestion réseau intelligente avec basculement automatique entre Wi-Fi et point d'accès, une interface web responsive hébergée sur Render pour visualiser les données en temps réel, des statistiques journalières, un historique exportable, et des notifications WhatsApp personnalisées via Twilio. Les tests ont validé la fiabilité du système, avec des mesures précises (par exemple, 24.8°C cohérent avec un thermomètre de référence), une transmission stable vers MongoDB, et une latence réseau acceptable (200 ms sur Eduroam). Les défis, tels que la configuration de WPA-Enterprise et la gestion des processus sous Linux, ont été surmontés grâce à une recherche approfondie et à l'utilisation d'outils comme **wpa_supplicant** et **systemd**.

En conclusion, ce projet a non seulement atteint ses objectifs initiaux de mesure et de transmission de température, mais les a dépassés en créant une solution autonome, connectée, et interactive, prête pour des applications réelles comme le monitoring environnemental dans un contexte universitaire ou professionnel. Il pourrait être enrichi à l'avenir par l'ajout de capteurs (humidité, pression) ou une intégration IoT plus large, tout en conservant sa robustesse et sa facilité d'utilisation. Ce travail illustre l'importance de l'adaptabilité et de l'innovation dans la conception de systèmes embarqués, compétences essentielles pour de futurs ingénieurs en électronique.

8. Annexes

8.1 Images des deux cartes Raspberry Pi reliés au PCB

Avec Raspberry Pi Pico 2 W :



Avec Raspberry Pi Zero 2 W :



- “raspi-gpio get” :

```

raspberrypi@raspberrypi:/sys/class/gpio $ raspi-gpio get
BANK0 (GPIO 0 to 27):
GPIO 0: level=1 func=INPUT
GPIO 1: level=1 func=INPUT
GPIO 2: level=1 func=INPUT
GPIO 3: level=1 func=INPUT
GPIO 4: level=1 func=INPUT
GPIO 5: level=1 func=INPUT
GPIO 6: level=1 func=INPUT
GPIO 7: level=1 func=OUTPUT
GPIO 8: level=1 func=OUTPUT
GPIO 9: level=0 alt=0 func=SPI0_MISO
GPIO 10: level=0 alt=0 func=SPI0_MOSI
GPIO 11: level=0 alt=0 func=SPI0_SCLK
GPIO 12: level=0 func=INPUT
GPIO 13: level=0 func=INPUT
GPIO 14: level=0 func=INPUT
GPIO 15: level=1 func=INPUT
GPIO 16: level=0 func=INPUT
GPIO 17: level=0 func=INPUT
GPIO 18: level=0 func=INPUT
GPIO 19: level=0 func=INPUT
GPIO 20: level=0 func=INPUT
GPIO 21: level=0 func=INPUT
GPIO 22: level=0 func=INPUT
GPIO 23: level=0 func=INPUT
GPIO 24: level=0 func=INPUT
GPIO 25: level=0 func=INPUT
GPIO 26: level=0 func=INPUT
GPIO 27: level=0 func=INPUT
BANK1 (GPIO 28 to 45):
GPIO 28: level=1 func=INPUT
GPIO 29: level=0 func=OUTPUT
GPIO 30: level=0 alt=3 func=CTS0
GPIO 31: level=0 alt=3 func=RTS0
GPIO 32: level=1 alt=3 func=TXD0
GPIO 33: level=1 alt=3 func=RXD0
GPIO 34: level=0 alt=3 func=SD1_CLK
GPIO 35: level=1 alt=3 func=SD1_CMD
GPIO 36: level=1 alt=3 func=SD1_DAT0
GPIO 37: level=1 alt=3 func=SD1_DAT1
GPIO 38: level=1 alt=3 func=SD1_DAT2
GPIO 39: level=1 alt=3 func=SD1_DAT3
GPIO 40: level=0 func=OUTPUT
GPIO 41: level=1 func=OUTPUT
GPIO 42: level=1 func=OUTPUT
GPIO 43: level=0 alt=0 func=GPCLK2
GPIO 44: level=1 func=INPUT
GPIO 45: level=1 func=INPUT
BANK2 (GPIO 46 to 53):
GPIO 46: level=1 func=INPUT
GPIO 47: level=1 func=OUTPUT

```


8.3 Pour connaître l'adresse IP de la carte Raspberry Pi Zero 2 W

Pour se connecter à la carte Raspberry Pi Zero 2 W avec RealVNC Viewer, vous devez connaître son adresse, pour cela utiliser **Angry IP Scanner**.

